

CS1006

Data Structures

Unit – I Abstract Data Types & Memory Allocation

After completing the course, the students will be able to

- CO1:** Understand and explore the concepts of Linear and Non-Linear Data Structures to develop solutions for diverse problems.
- CO2:** Apply the concepts of Hashing, Sorting and Searching techniques to get an Optimized solution.
- CO3:** Evaluate and apply appropriate data structure(s) for real-world problems.
- CO4:** Demonstrate good coding practices involving lifelong learning.

Module – 1 Abstract Data Types & Memory Allocation

9 Hours

Strategies for choosing the appropriate data structure. Abstract data types. Stack Memory - Compile Time, Static memory allocation. Heap Memory - Runtime, dynamic memory allocation. Arrays – Revision of basic operations. Implementation of basic operations – Create, Insert, Traverse, Delete, Search and Update.

Module – 2 Linked List

9 Hours

Linked lists – Types – Singly Linked List – Doubly Linked List – Circular Linked List -(Circular Singly, Circular Doubly). Implementation of operations – Create, Insert, Traverse, Delete, Search and Update.

Module – 3 Stacks and Queues **Hours**

9

Introduction to Stacks and Queues. Implementation of their basic operations. Use basic arrays & link list to create stack & queue data structures.

Module – 4 Sorting, Searching, and Hashing

9 Hours

Linear Search, Binary Search, Bubble Sort, Selection Sort, Insertion Sort. Hashing – Hash tables, has functions, strategies to avoid and resolve collisions.

Module – 5 Trees

9 Hours

Binary Tree, Binary Search Tree, AVL Tree, B-Trees, Common operations on these trees such as select min, select max, create, insert, delete. Perform various types of Tree Traversals.

Data Structure – Basics

- ❖ Data structures are generally based on the ability of a computer to fetch and store data at any place in its memory, specified by a pointer.
- ❖ Thus, the array and record data structures are based on computing the addresses of data items with arithmetic operations

Data Structures + Algorithms = Program

- Program -> A Set of Instructions
- Data Structure -> A Container stores Data
- Algorithm -> Logic + Control

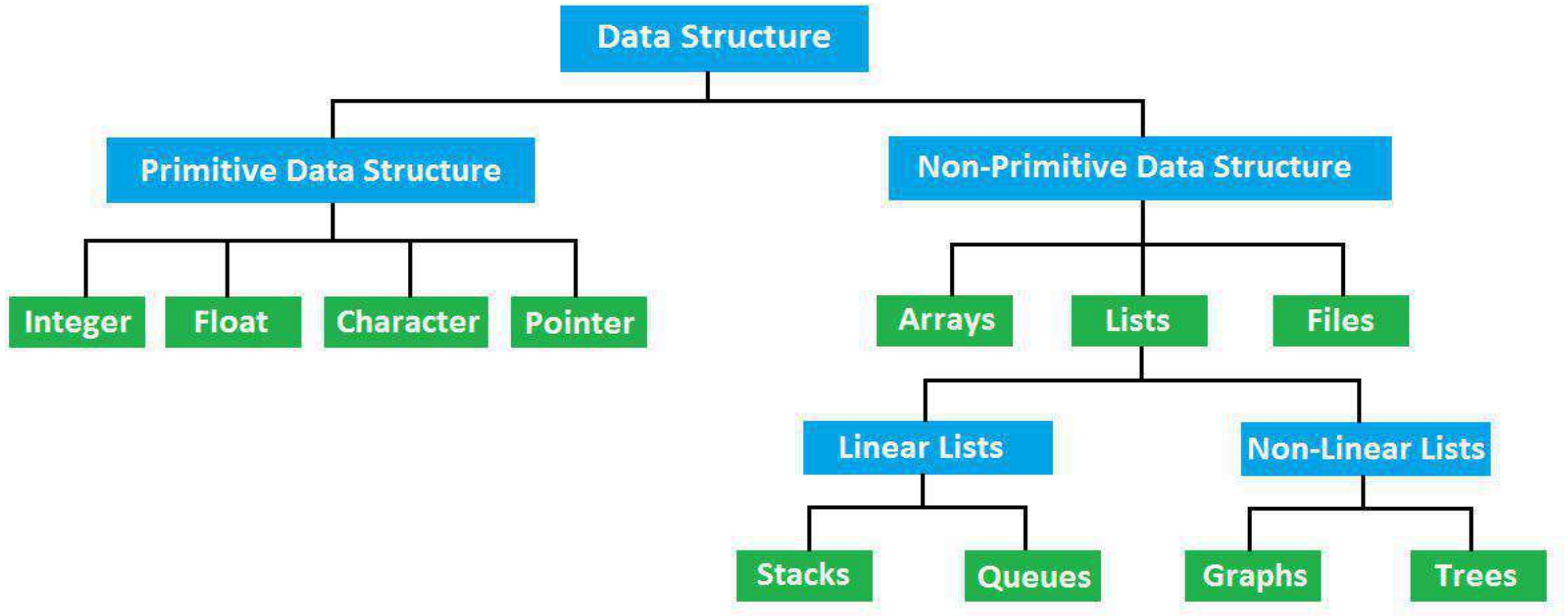
Data Structure – Basics

- Data: simply a value or set of values of different types which is called data types like string, integer, char etc.
- Structure: Way of organizing information, so that it is easier to use.

Data Structure ..

- ❖ A data structure is a particular way of organizing data in a computer so that it can be used efficiently.
- ❖ A scheme for organizing related pieces of information.

Data Structures – Classification



Definitions

Primitive Data Structure:

- ✓ A Primitive Data Structure used to represent the standard data types of any one of the computer languages (integer, Character, float etc.).

Non - Primitive Data Structure :

- ✓ A Non - Primitive Data Structure can be constructed with the help of any one of the primitive data structure.
 - It can be structured having a specific functionality.
 - It can be designed by user.
 - It can be classified as Linear and Non-Linear Data Structure.

□ Linear Data Structures:

- A linear data structure traverses the data elements sequentially, in which only one data element can directly be reached.
- Ex: Arrays, Linked Lists ,Stacks, Queues

□ Non-Linear Data Structures:

- Every data item is attached to several other data items in a way that is specific for reflecting relationships. The data items are not arranged in a sequential structure.
- Ex: Trees, Graphs ,Heaps

Basic Operations on Data structures

□ The basic operations that are performed on data structures are as follows:

- 1. Traversal:** Traversal of a data structure means processing all the data elements present in it exactly once.
- 2. Insertion:** Insertion means addition of a new data element in a data structure.
- 3. Deletion:** Deletion means removal of a data element from a data structure if it is found.

Basic Operations on Data structures

4. Searching: Searching involves searching for the specified data element in a data structure.

5. Sorting: Arranging data elements of a data structure in a specified order is called sorting.

6. Merging: Combining elements of two similar data structures to form a new data structure of the same type, is called merging.

Abstract Data Type (ADT)

- ✓ An Abstract Data type refers to set of data values and associated operations that are specified accurately, independent of any particular implementation. (Or)
- ✓ ADT is a user defined data type which encapsulates a range of data values and their functions. (Or)
- ✓ An **Abstract Data Type** is a mathematical model of a **data structure**. It describes a container which holds a finite number of objects where the objects may be associated through a given binary relationship.
- ✓ Ex: List, Stack ,Queue

The List ADT

- ✓ A **list** or **sequence** is an abstract data type that represents a countable number of ordered values, where the same value may occur more than once.
- ❖ A sequence of zero or more elements $A_1, A_2, A_3, \dots A_N$
- ❖ length of the list A_1 : first element A_N : last element A_i : position i
- ❖ If $N=0$, then empty list
- ❖ Linearly ordered $\rightarrow A_i$ precedes A_{i+1} (or) A_i follows A_{i-1}
- ✓ Lists are a basic example of containers, as they contain other values. If the same value occurs multiple times, each occurrence is considered a distinct item.

The List ADT – Operations

- ✓ A list contains elements of same type arranged in sequential order and following operations can be performed on the list.
- **get()** – Return an element from the list at any given position.
- **insert()** – Insert an element at any position of the list.
- **remove()** – Remove the first occurrence of any element from a non-empty list.

The List ADT – Operations

- `removeAt()` – Remove the element at a specified location from a non-empty list.
- `replace()` – Replace an element at any position by another element.
- `size()` – Return the number of elements in the list.
- `isEmpty()` – Return true if the list is empty, otherwise return false.
- `isFull()` – Return true if the list is full, otherwise return false.

Implementation of List ADT

- ❖ Each operation associated with the ADT is implemented by one or more subroutines.
- ❖ Two standard implementations for the list ADT
 - ✓ Array-based
 - ✓ Linked list
- ❖ An array is a *random access* data structure, where each element can be accessed directly and in constant time.
 - A typical illustration of random access is a book - each page of the book can be open independently of others.

Memory Allocation



Unit 1: Array and Linked List

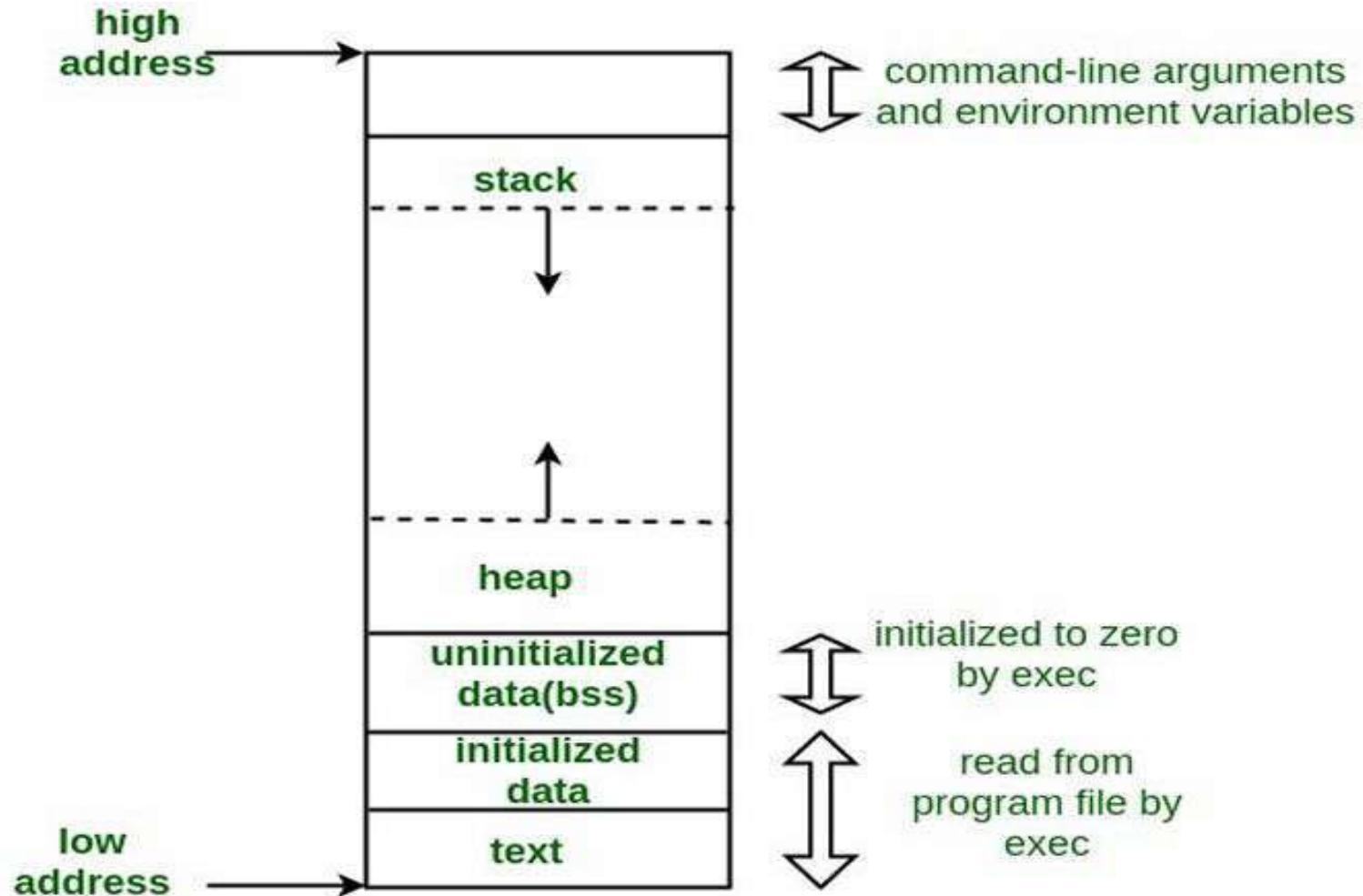
12 hours

- Strategies for choosing the appropriate data structure,
- Abstract Data types,
- **Stack Memory – Compile time,**
- **Static Memory Allocation,**
- **Heap Memory – Runtime ,**
- **Dynamic Memory Allocation.**
- Arrays
- Revision of Basic Operations
- ~~Linked Lists (Singly, Doubly, Circular Singly, Circular Doubly).~~
- ~~Implementation of Operations – Create, Insert, Traverse, Delete, Search and Update.~~

Memory Layout in 'C' Program

- After compiling a C program, a binary executable file(.exe) is created, and when we execute the program, this binary file loads into RAM in an organized manner.
- After being loaded into the RAM, memory layout in C Program has six components which are **text segment, initialized data segment, uninitialized data segment, command-line arguments, stack, and heap**.
- Each of these six different segments stores different parts of code and have their own read, write permissions.
- If a program tries to access the value stored in any segment differently than it is supposed to, it results in a **segmentation fault error**.

Memory Layout in 'C' Program



Text Segment

- After we compile the program, a binary file generates, which is used to execute our program by loading it into RAM. This binary file contains instructions, and these **instructions get stored in the text segment of the memory**.
- Text segment has **read-only permission** that prevents the program from accidental modifications.
- Text segment in RAM is **shareable** so that a single copy is required in the memory for frequent applications like a text editor, shells, etc.

Initialized Data Segment

- Initialized data segment or data segment is part of the computer's virtual memory space of a C program that **contains values of all external, global, static, and constant variables** whose values are initialized at the time of variable declaration in the program.
- Because the values of variables can change during program execution, this memory segment has **read-write permission**.
- We can further classify the data segment into the **read-write and read-only areas**.
- *const* variable comes under the read-only area

Program

```
#include<stdio.h>

/* global variables stored in the read-write part of
   initialized data segment
   */
int global_var = 50;
char* hello = "Hello World";
/* global variables stored in the read-only part of
   initialized data segment
   */
const int global_var2 = 30;

int main() {
    // static variable stored in initialized data segment
    static int a = 10;
    // ...
    return 0;
}
```


Uninitialized Data Segment

- An uninitialized data segment is also known as **bss** (**b**lock **s**tarted by **s**ymbol).
- The program loaded allocates memory for this segment when it loads. **Every data in bss is initialized to arithmetic 0 and pointers to null pointer** by the kernel before the C program executes.
- BSS also **contains all the static and global variables, initialized with arithmetic 0.**
- Because values of variables stored in bss can be changed, this data segment has **read-write permissions**

Practice Questions

- Write a program to initialize four pointers of primitive data type (char, integer, float, double).

Use 'sizeof' operator and print the amount of memory occupied by a pointer to a primitive data type (char, integer, float, double).

- Write a program to initialize four arrays of primitive data type (char, integer, float, double).

Hint: You must create an integer array, char array, float array and double array of size 5.

Use pointer to an array for each of the four arrays and,

Use 'sizeof' operator and print the amount of memory occupied by a pointer to an array holding a primitive data type (char, integer, float, double).

Use 'sizeof' operator and print the amount of memory occupied by an array holding primitive data types (char, integer, float, double)

Stack Memory

- The stack memory makes use of the stack data structure.
- Stack memory is a memory usage mechanism that allows the system memory to be used as temporary data storage that behaves as a LIFO (Last In First Out) buffer.
- Stack segment stores the value of **local variables and values of parameters passed to a function** (by creating stack frame for each function) along with some additional information like the instruction's **return address**, which is to be executed after a function call.
- Stack grows in the direction **opposite to heap**.


```
#include<stdio.h>
int total;
int Square(int x)
{
    return x*x;
}
int SquareOfSum(int x,int y)
{
    int z = Square(x+y);
    return z;
}
int main()
{
    int a = 4, b = 8;
    total = SquareOfSum(a,b);
    printf("output = %d",total);
}
```

- One of the common cases of stack overflow is when you write basic recursive programs. Your program may go into infinite loop.
- Memory set aside for stack doesn't grow during run time.
- Application cannot request more memory from stack.
- Allocation and deallocation of memory happens using a set of rules.
- It is not possible to manipulate the scope of the variable if it is on stack.

Heap Memory

- Unlike stack, applications **heap memory is not fixed**.
- It can change during run time.
- Heap is used for memory which is **allocated during the run time** (dynamically allocated memory).
- Heap is referred to as dynamic memory and using heap for program/application is referred to as **dynamic memory allocation**.
- Heap is also called as the **free pool of memory**.
- To use dynamic memory in C we need to know about four functions: **malloc(), calloc(), free(), realloc()**. These four functions are used to manage allocations in the heap segment

Types of Memory Allocation in 'C'

C programming or any other programming language basically support two types of memory allocation.

1. Compile time memory allocation
2. Runtime memory allocation

Compile Time Memory Allocation

- Compile time memory allocation means reserving memory for variables, constants **during the compilation process**.
- The memory allocated is **fixed and cannot be increased or decreased** during run time.
- So, you must exactly know how many bytes you require?
- Many text also refer compile time memory allocation as **static or stack memory allocation**.
- In static memory allocation, once the memory is allocated, the memory size can not change

Key Features of Static Memory Allocation

- Allocation and deallocation are **done by the compiler**.
- It **uses** a data structures **stack** for static memory allocation.
- Variables get **allocated permanently**.
- **No reusability**.
- Execution is **faster** than dynamic memory allocation.
- Memory is allocated before runtime.
- It is **less efficient**

Run Time Memory Allocation

- Memory allocation done at the time of execution(run time) is known as dynamic memory allocation.
- **Heap** is referred to as dynamic memory. Making use of heap for allocation of memory is known as dynamic memory allocation.

NOTE: Not be to confused with heap data structure.

- Also sometimes known as free pool of memory.

Run Time Memory Allocation

- **Pointers play an important role** in Dynamic Memory Allocation.
- Only way to access memory on heap is through pointers.
- To use dynamic memory in C we need four built-in functions:

✓ malloc()

✓ calloc()

✓ realloc()

✓ free()

Key Features

- We can also **reallocate memory size** if needed.
- Dynamic Allocation is **done at run time**.
- **No memory wastage**

malloc()

- The “malloc” or “memory allocation” method in C is used to dynamically allocate a **single large block of memory** with the specified size.
- It **returns a pointer of type void** which can be cast into a pointer of any form.
- It initializes **each block** with the **default garbage value** initially.

Syntax: `ptr = (cast-type*) malloc(byte-size)`

malloc() – Example

Case 1.

```
ptr = (int*) malloc(1 * sizeof(int));
```

Since the size of int is 4 bytes, this statement will allocate 4 bytes of memory.

Case 2.

```
ptr = (int*) malloc(100 * sizeof(int));
```

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And the pointer ptr holds the address of the first byte in the allocated memory

Malloc()

```
int* ptr = ( int* ) malloc ( 5* sizeof ( int ) );
```

Diagram illustrating the memory allocation process:

- The expression `sizeof ( int )` is annotated with a bracket and labeled "4 bytes", indicating the size of an integer.
- The variable `ptr` is shown pointing to a rectangular box representing the allocated memory.
- A horizontal double-headed arrow below the box is labeled "20 bytes of memory", indicating the total size of the allocated block.
- An arrow points from the text "A large 20 bytes memory block is dynamically allocated to ptr" to the memory box.



“calloc” or “contiguous allocation” method in C is used to dynamically allocate the **specified number of blocks** of memory of the specified type.

- It is very similar to malloc() but has two different points and these are:

1. It **initializes each block with** a default value ‘0’.
2. It has two parameters or arguments as compared to malloc().

Syntax: `ptr = (cast-type*)calloc(n, element-size);`

‘n’ is the no. of elements and element-size is the size of each element.

calloc() – Example

For Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

This statement allocates contiguous space in memory for 25 elements each with the size of the float

Calloc()

```
int* ptr = ( int* ) calloc ( 5, sizeof ( int ) );
```

Diagram illustrating the memory allocation for the above code. A yellow arrow points from the number '5' in the code to a diagram of five adjacent rectangular blocks. Another yellow arrow points from the text 'sizeof (int)' in the code to the text '4 bytes'.



realloc()

- “realloc” or “re-allocation” method in C is used to dynamically change the memory allocation of a previously allocated memory.
- In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**.
- re-allocation of memory maintains the already present value and **new blocks will be initialized with the default garbage value**.

Syntax: `ptr = realloc(ptr, newSize);`

- where ptr is reallocated with new size 'newSize'

Realloc()

```
int* ptr = ( int* ) malloc ( 5* sizeof ( int ) );
```

ptr =



← 20 bytes of memory →

A large 20 bytes memory block is dynamically allocated to ptr

```
ptr = realloc ( ptr, 10* sizeof( int ) );
```

ptr =



← 40 bytes of memory →

The size of ptr is changed from 20 bytes to 40 bytes dynamically



- “free” method in C is used to dynamically de-allocate the memory.
- The memory allocated using functions malloc() and calloc() is not de allocated on their own.
- Hence the free() method is used, whenever the dynamic memory allocation takes place.
- It helps to reduce wastage of memory by freeing it.

Syntax: free(ptr);

Free()

```
int* ptr = ( int* ) calloc ( 5, sizeof ( int ) );
```

ptr =



4 bytes

5 blocks of 4 bytes each is dynamically allocated to ptr

operation on ptr

free(ptr)



The memory of ptr is released



GG

Comparision

Compile Time Memory Allocation	Run Time Memory Allocation
Memory is allocated during program compilation	Memory is allocated during runtime
Cannot reuse allocated memory	Can reuse allocated memory after releasing memory using free() function
Library functions are not required to allocate memory	Requires library functions like malloc(), calloc(), realloc() to allocate memory at runtime.
Cannot modify size of allocated memory	Can modify size of allocated memory using realloc()

Arrays & its operations



Unit I

9 hours

- Strategies for choosing the appropriate data structure,
- Abstract Data types,
- Stack Memory – Compile time,
- Static Memory Allocation,
- Heap Memory – Runtime ,
- Dynamic Memory Allocation.
- **Arrays**
- **Revision of Basic Operations**

Array Based Implementation of List

- An Array is a data structure which can store a fixed-size sequential collection of elements of the same type.
- Array declaration use `numbers[0]`, `numbers[1]`,..., `numbers[99]` to represent individual variables.
- A specific element in an array is accessed by an index.
- All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

Array Based Implementation of List



List Structure

Index	Array	Position
List [0]		1
List [1]		2
List [2]		3
List [3]		4
List [4]		5

Array Name: **List**
List Size : **5**
Start Position: **1**
End Position: **5**
Start Index: **0 i.e List[0]**
End Index; **4 i.e List[4]**
1st Element referred by: **List[0]**
5th Element referred by: **List[4]**
 j^{th} Element referred by: **List[i-1]**

Variable	Value
Max Size	5
Current Size	0

Max Size: Number of elements we can insert to the List.
Current Size: Number of elements already inserted to the List

Is Empty(LIST)

- If (Current Size==0) "LIST is Empty"
- else "LIST is not Empty"

Is Full(LIST)

- If (Current Size=Max Size) "LIST is FULL"
- else "LIST is not FULL"

Insert Element to End of the LIST

- Check whether the List is full or not.

✓ If List is full return error message "List is full. Can't Insert".

✓ If List is not full.

❖ Get the position to insert the new element by

$\text{Position} = \text{Current Size} + 1$

❖ Insert the element to the Position

❖ Increase the Current Size by 1 i.e. $\text{Current Size} = \text{Current Size} + 1$

Delete Element from End of the LIST

- Check that whether the List is empty or not
 - If List is empty return error message "List is Empty. Can't Delete".
 - If List is not Empty.
- ✓ Get the position of the element to delete by $\text{Position} = \text{Current Size}$
 - ✓ Delete the element from the Position
 - ✓ Decrease the Current Size by 1 i.e. $\text{Current Size} = \text{Current Size} - 1$

Insert Element to front of the LIST

- Check that whether the List is full or not
 - If List is full return error message "List is full. Can't Insert".
 - If List is not full.
- ✓ Free the 1st Position of the list by moving all the Element to one position forward i.e $\text{New Position} = \text{Current Position} + 1$.
 - ✓ Insert the element to the 1st Position
 - ✓ Increase the Current Size by 1 i.e. $\text{Current Size} = \text{Current Size} + 1$

Delete Element from front of the LIST

- Check that whether the List is empty or not
- If List is empty return error message "List is Empty. Can't Delete".
- If List is not Empty
 - ✓ Move all the elements except one in the 1st position to one position backward i.e $\text{New Position} = \text{Current Position} - 1$
 - ✓ After the 1st step, element in the 1st position will be automatically deleted. Decrease the Current Size by 1.
 - ✓ (i.e.) $\text{Current Size} = \text{Current Size} - 1$

Insert Element to nth Position of the LIST

- Check whether the List is full or not
- If List is full return error message "List is full. Can't Insert".
- If List is not full.

- ✓ If List is Empty, Insert element at Position 1.
- ✓ If (nth Position > Current Size)
- ✓ Return message "nth Position Not available in List" else
- ✓ Free the nth Position of the list by moving all Elements to one position forward except n-1,n-2, ... 1 Position (i.e) move only from n to current size position Elements. i.e New Position=Current Position + 1.
- ✓ Insert the element to the nth Position.
- ✓ Increase the Current Size by 1 i.e. Current Size=Current Size+1

Delete Element from nth Position of the LIST

- Check that whether the List is Empty or not
 - If List is Empty return error message "List is Empty."
 - If List is not Empty.
- ✓ If (nth Position > Current Size)
 - ✓ Return message "nth Position Not available in List"
 - ✓ If (nth Position == Current Size)
 - ✓ Delete the element from nth Position
 - ✓ Decrease the Current Size by 1 i.e. $\text{Current Size} = \text{Current Size} - 1$
 - ✓ If (nth Position < Current Size)
 - ✓ Move all the Elements to one position backward except n, n-1, n-2, ... 1 Position i.e move only from n+1 to current size position Elements. i.e $\text{New Position} = \text{Current Position} - 1$.
 - ✓ After the previous step, nth element will be deleted automatically.
 - ✓ Decrease the Current Size by 1 i.e. $\text{Current Size} = \text{Current Size} - 1$

Search Element in the LIST

- Check that whether the list is empty or not.
- If List is empty, return error message “List is Empty”.
- If List is not Empty

- ✓ Find the Position where the last element available in the List by
 - ✓ Last Position = Current Size
 - ✓ For(Position 1 to Last Position)
 - ✓ If(Element @ Position== Search Element) //If Element matches the search element
 - ✓ return the Position by message “Search Element available in Position”
 - ✓ Else return message “Search Element not available in the List”

Print the Elements in the LIST

- Check that whether the list is empty or not.
 - If List is empty, return error message “List is Empty”.
 - If List is not Empty
- ✓ Find the Position where the last element available in the List by
 - ✓ Last Position = Current Size
 - ✓ For(Position 1 to Last Position)
 - ✓ Print the Position and Element available at the position of List.

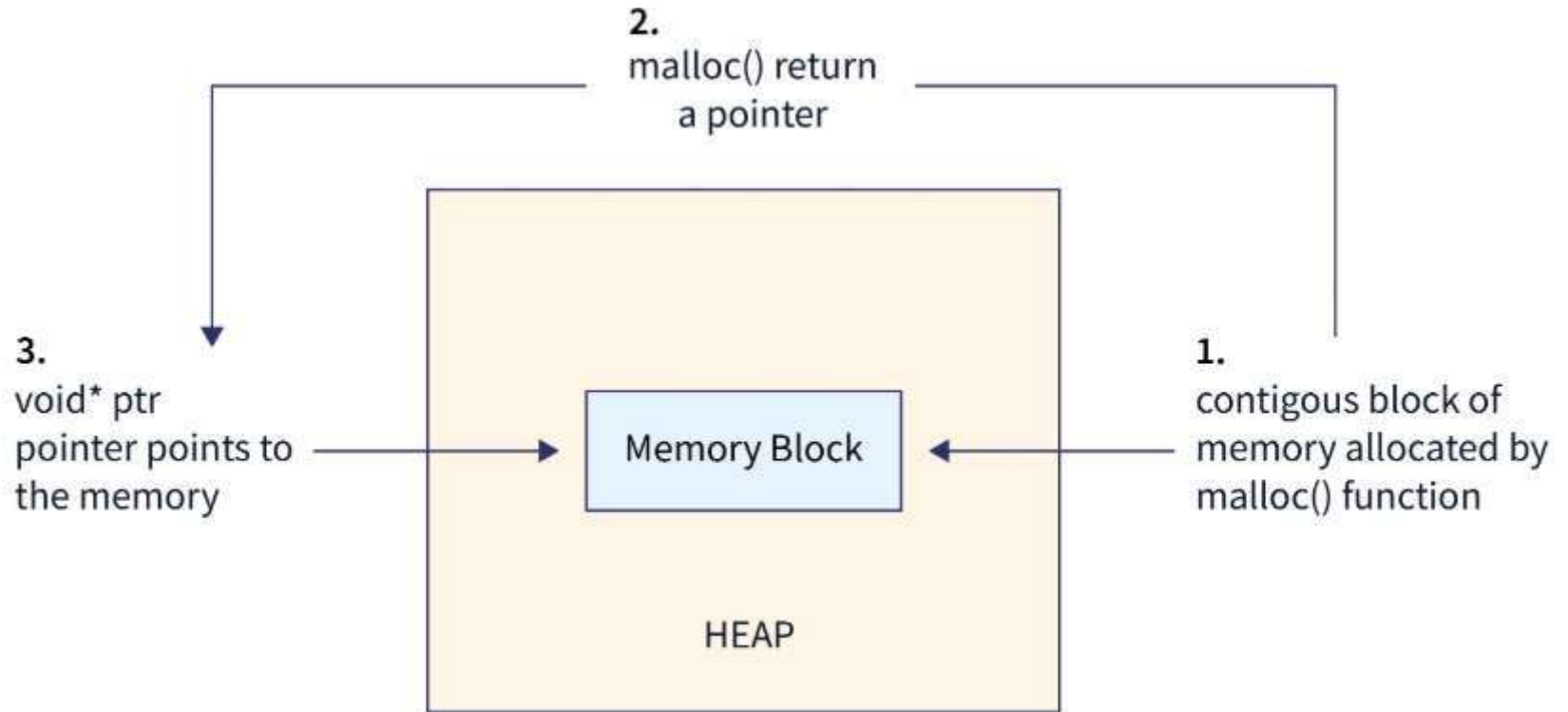
External References

1. <https://nptel.ac.in/courses/106102064/>
2. <https://nptel.ac.in/courses/106103069/>
3. <https://nptel.ac.in/courses/106106127/>
4. <https://nptel.ac.in/courses/106106130/>
5. <https://www.geeksforgeeks.org/data-structures/>
6. <https://www.javatpoint.com/data-structure-tutorial>
7. <https://www.coursera.org/learn/data-structures>
8. <https://www.studytonight.com/data-structures/introduction-to-data-structures>
9. <https://www.youtube.com/watch?v=egb6qFw42JM> (Tamil videos)

Thank You

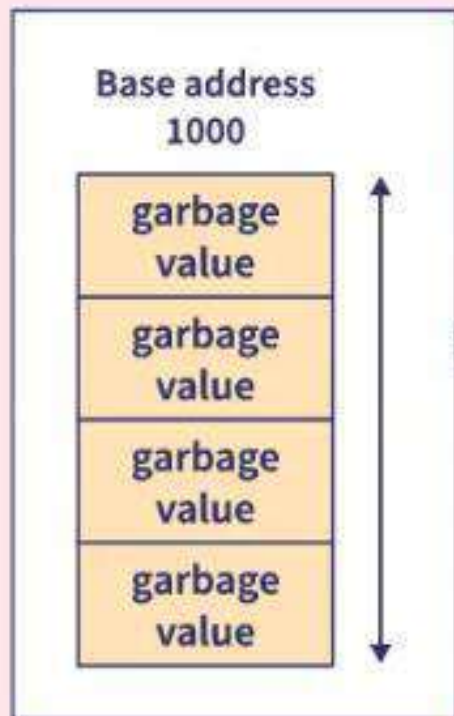
Module-1 Additional Information

Difference between malloc() and calloc() :



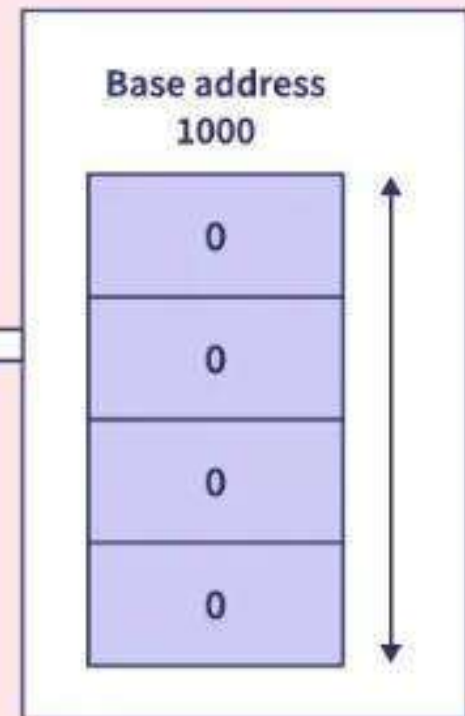
Difference between malloc() and calloc() :

malloc



VS

calloc



```
malloc(5 * sizeof(int))
```

```
+-----+-----+-----+-----+-----+
|  ??  |  ??  |  ??  |  ??  |  ??  |
+-----+-----+-----+-----+-----+
```

```
calloc(5, sizeof(int))
```

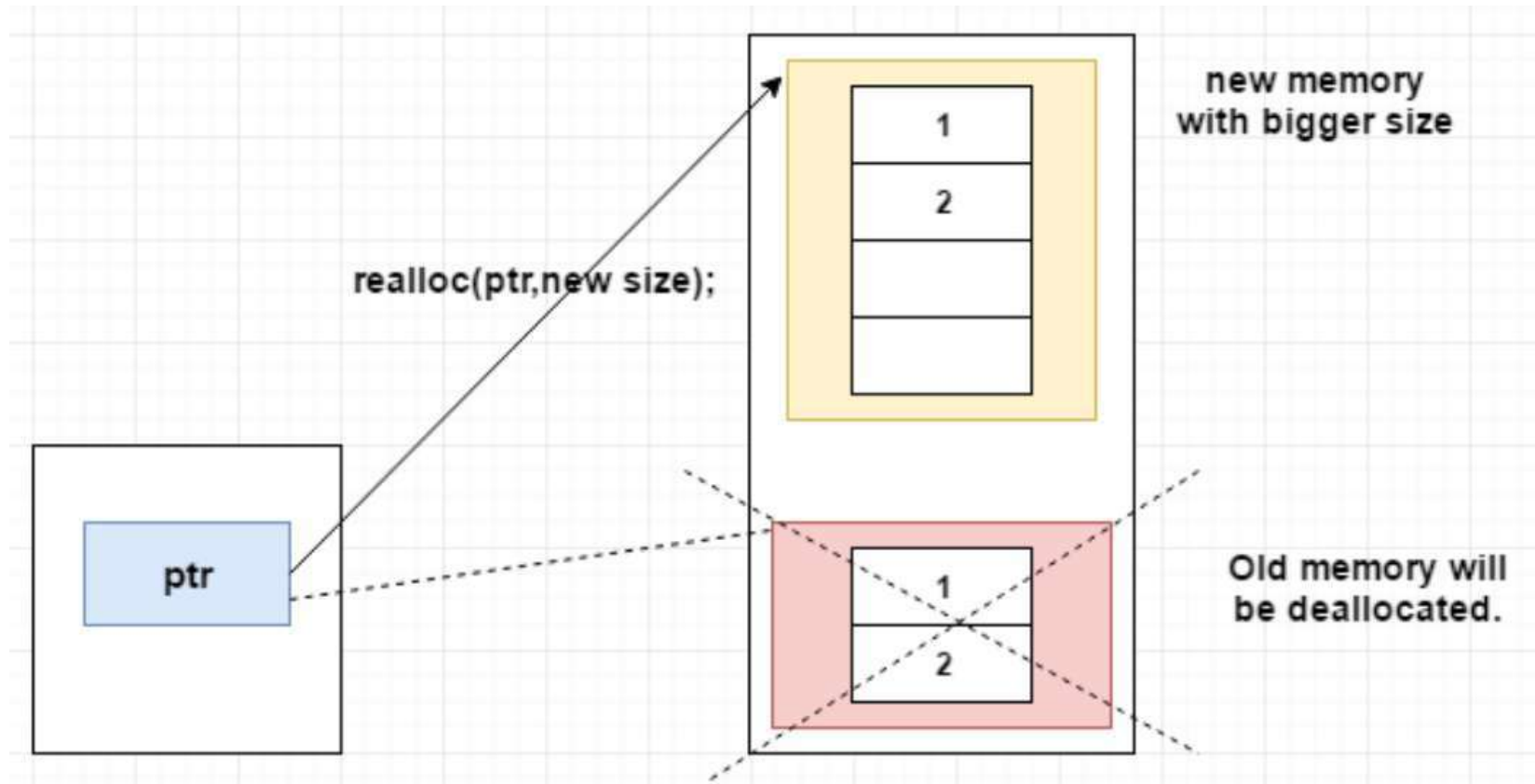
```
+-----+-----+-----+-----+-----+
|  0   |  0   |  0   |  0   |  0   |
+-----+-----+-----+-----+-----+
```




Quick Comparison Table (Exam-friendly)

Feature	malloc()	calloc()
Full form	Memory Allocation	Contiguous Allocation
Initialization	✗ No (garbage)	✓ Yes (0)
Arguments	1	2
Speed	Faster	Slightly slower
Memory layout	Single block	Contiguous blocks

realloc()



Case 1: Expanding memory (space available nearby)

Before `realloc()`

```
sql  
  
malloc(3 * sizeof(int))
```

```
+-----+  
| 10 | 20 | 30 |  
+-----+
```

After `realloc(ptr, 5 * sizeof(int))`

```
diff  
  
+-----+  
| 10 | 20 | 30 | ?? | ?? |  
+-----+
```

- ✓ Old data is **preserved**
- ✓ Extra memory is **uninitialized**

Case 2: Expanding memory (no adjacent space → new location)

```
Old Block          New Block  
+-----+         +-----+  
| 10 | 20 | 30 | --> | 10 | 20 | 30 | ?? | ?? |  
+-----+         +-----+  
      ^ old freed
```

- ✓ Data is **copied**
- ✓ Old memory is **freed automatically**
- ✓ Pointer may **change address**

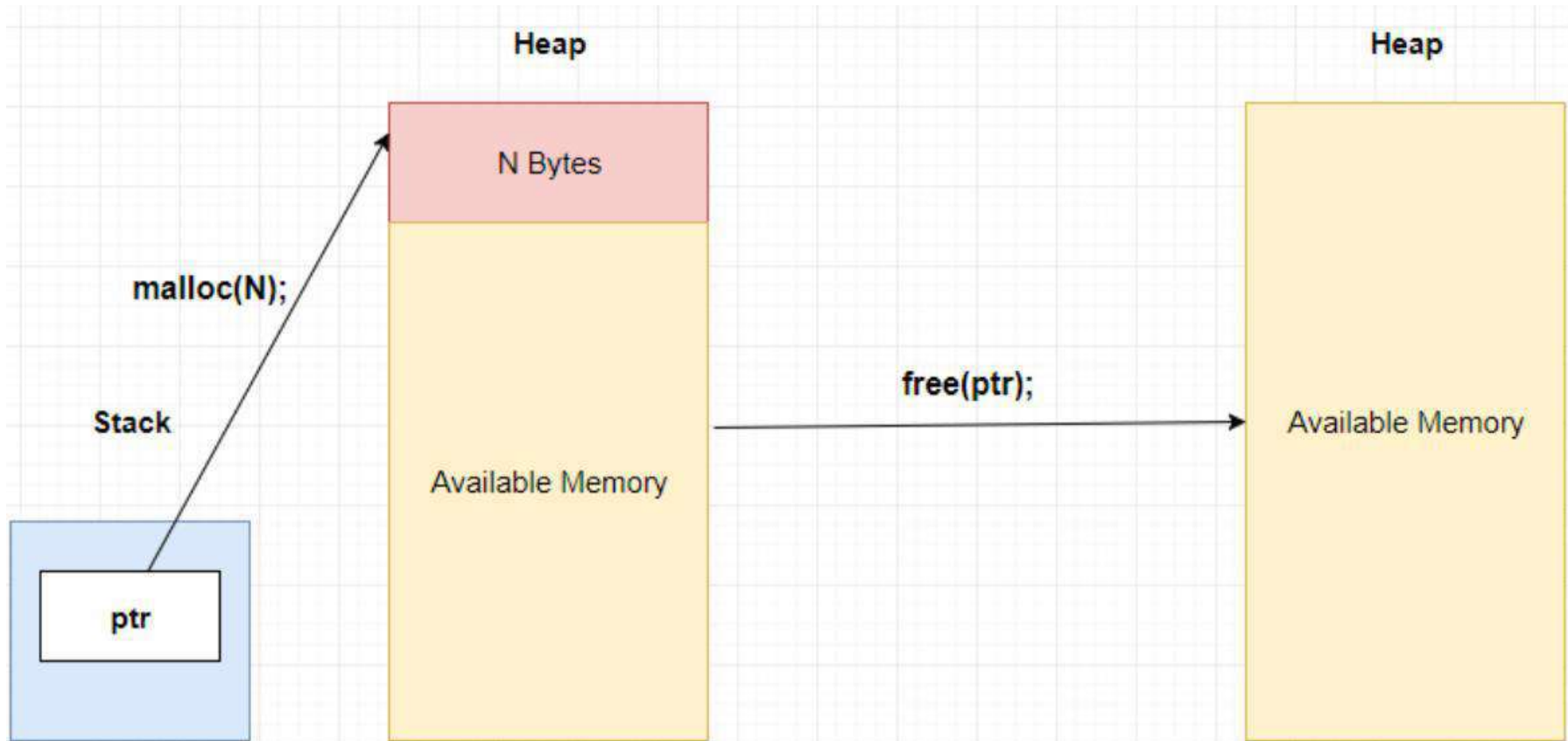
Case 3: Shrinking memory

```
realloc(ptr, 2 * sizeof(int))
```

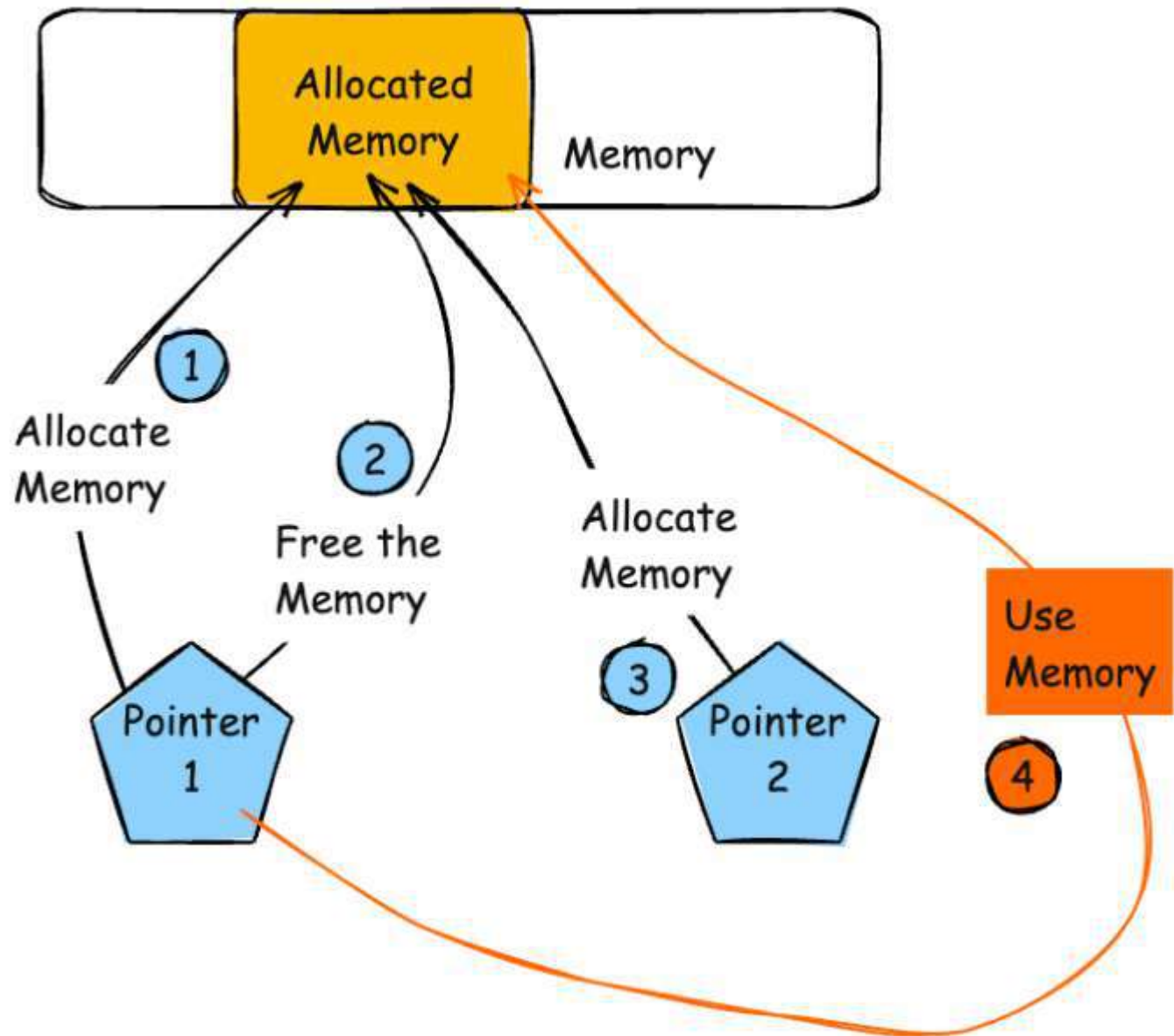
```
+---+---+  
| 10 | 20 |  
+---+---+
```

- ✓ Extra memory is released
- ✓ Initial elements are retained

free()



`free()`



free()

```
free(ptr);
```

ptr

|



+---+---+---+---+

| freed memory |

+---+---+---+---+

- ✓ Memory returned to **heap**
- ✗ Pointer becomes a **dangling pointer**

//C program for PRACTICE PROBLEM-1

```
#include <stdio.h>
```

```
int main() {
```

```
    char c = 'A';
```

```
    int i = 10;
```

```
    float f = 3.14f;
```

```
    double d = 9.876;
```

```
    /* Pointer initialization */
```

```
    char *pc = &c;
```

```
    int *pi = &i;
```

```
    float *pf = &f;
```

```
    double *pd = &d;
```

```
    /* Printing size of pointers */
```

```
    printf("Size of char pointer   : %zu bytes\n", sizeof(pc));
```

```
    printf("Size of int pointer    : %zu bytes\n", sizeof(pi));
```

```
    printf("Size of float pointer   : %zu bytes\n", sizeof(pf));
```

```
    printf("Size of double pointer  : %zu bytes\n", sizeof(pd));
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
//C program for PRACTICE PROBLEM-2
```

```
int main() {
```

```
    /* Array initialization */
```

```
    char cArr[5] = {'A', 'B', 'C', 'D', 'E'};
```

```
    int iArr[5] = {1, 2, 3, 4, 5};
```

```
    float fArr[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
```

```
    double dArr[5] = {10.1, 20.2, 30.3, 40.4, 50.5};
```

```
    /* Pointer to array */
```

```
    char (*pc)[5] = &cArr;
```

```
    int (*pi)[5] = &iArr;
```

```
    float (*pf)[5] = &fArr;
```

```
    double (*pd)[5] = &dArr;
```

```
    /* Size of pointers to arrays */
```

```
    printf("Size of pointer to char array : %zu bytes\n", sizeof(pc));
```

```
    printf("Size of pointer to int array : %zu bytes\n", sizeof(pi));
```

```
    printf("Size of pointer to float array : %zu bytes\n", sizeof(pf));
```

```
    printf("Size of pointer to double array : %zu bytes\n\n", sizeof(pd));
```



```
/* Size of arrays */  
printf("Size of char array : %zu bytes\n", sizeof(cArr));  
printf("Size of int array : %zu bytes\n", sizeof(iArr));  
printf("Size of float array : %zu bytes\n", sizeof(fArr));  
printf("Size of double array : %zu bytes\n", sizeof(dArr));  
  
return 0;  
}
```

//C program to Insert Element to End of the LIST

```
#include <stdio.h>
#define MAX 5 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, element, position, i;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Read existing elements
    printf("Enter %d elements:\n", currentSize);
    for (i = 0; i < currentSize; i++)
    {
        scanf("%d", &list[i]);
    }

    // Check whether the list is full
    if (currentSize == MAX)
    {
        printf("List is full. Can't Insert\n");
        return 0;
    }
}
```

```
// Read new element to be inserted
printf("Enter element to insert: ");
scanf("%d", &element);

// Get position to insert (end of the list)
position = currentSize + 1;

// Insert element at the end
list[position - 1] = element;

// Increase current size
currentSize = currentSize + 1;

// Display updated list
printf("List after insertion:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}

return 0;
}
```


//C Program to Delete Element from End of the LIST

```
#include <stdio.h>
#define MAX 5 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, position, i;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Check whether the list is empty
    if (currentSize == 0)
    {
        printf("List is Empty. Can't Delete\n");
        return 0;
    }
}
```

```
// Read elements of the list
printf("Enter %d elements:\n", currentSize);
for (i = 0; i < currentSize; i++)
{
    scanf("%d", &list[i]);
}

// Get position of element to delete (end position)
position = currentSize;

// Delete element from the end
// (No shifting required for deletion at end)

// Decrease current size
currentSize = currentSize - 1;

// Display list after deletion
printf("List after deletion:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}
return 0;
}
```

```
#include <stdio.h>
```

```
#define MAX 5 // Maximum size of the list
```

```
int main()
```

```
{
```

//C Program to Insert Element to front of the LIST

```
    int list[MAX];
```

```
    int currentSize, element, i;
```

```
    // Read current size of the list
```

```
    printf("Enter current size of the list: ");
```

```
    scanf("%d", &currentSize);
```

```
    // Read elements of the list
```

```
    printf("Enter %d elements:\n", currentSize);
```

```
    for (i = 0; i < currentSize; i++)
```

```
    {
```

```
        scanf("%d", &list[i]);
```

```
    }
```

```
    // Check whether the list is full
```

```
    if (currentSize == MAX)
```

```
    {
```

```
        printf("List is full. Can't Insert\n");
```

```
        return 0;
```

```
    }
```



```
// Read element to be inserted
printf("Enter element to insert at front: ");
scanf("%d", &element);

// Free the first position by shifting elements one position forward
for (i = currentSize - 1; i >= 0; i--)
{
    list[i + 1] = list[i];
}

// Insert element at the first position
list[0] = element;

// Increase current size
currentSize = currentSize + 1;

// Display updated list
printf("List after insertion at front:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}
return 0;
}
```

// C Program to Delete Element from front of the LIST

```
#include <stdio.h>
```

```
#define MAX 5 // Maximum size of the list
```

```
int main()
```

```
{
```

```
    int list[MAX];
```

```
    int currentSize, i;
```

```
    // Read current size of the list
```

```
    printf("Enter current size of the list: ");
```

```
    scanf("%d", &currentSize);
```

```
    // Check whether the list is empty
```

```
    if (currentSize == 0)
```

```
    {
```

```
        printf("List is Empty. Can't Delete\n");
```

```
        return 0;
```

```
    }
```

```
// Read elements of the list
printf("Enter %d elements:\n", currentSize);
for (i = 0; i < currentSize; i++)
{
    scanf("%d", &list[i]);
}

// Move all elements one position backward
for (i = 1; i < currentSize; i++)
{
    list[i - 1] = list[i];
}

// Decrease current size
currentSize = currentSize - 1;

// Display list after deletion
printf("List after deletion from front:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}
return 0;
}
```


//C program to Insert Element to nth Position of the LIST

```
#include <stdio.h>
#define MAX 10 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, element, position, i;
    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);
    // Check whether the list is full
    if (currentSize == MAX)
    {
        printf("List is full. Can't Insert\n");
        return 0;
    }
    // Read elements if list is not empty
    if (currentSize > 0)
    {
        printf("Enter %d elements:\n", currentSize);
        for (i = 0; i < currentSize; i++)
        {
            scanf("%d", &list[i]);
        }
    }
}
```

```
// Read position (n) to insert
printf("Enter the position to insert: ");
scanf("%d", &position);
// Read element to insert
printf("Enter element to insert: ");
scanf("%d", &element);
// If list is empty, insert at position 1
if (currentSize == 0 && position == 1)
{
    list[0] = element;
    currentSize = currentSize + 1;
}
// If position is greater than current size + 1
else if (position > currentSize + 1)
{
    printf("nth Position Not available in List\n");
    return 0;
}
```

```
else
{
    // Free the nth position by shifting elements forward
    for (i = currentSize - 1; i >= position - 1; i--)
    {
        list[i + 1] = list[i];
    }
    // Insert element at nth position
    list[position - 1] = element;

    // Increase current size
    currentSize = currentSize + 1;
}

// Display updated list
printf("List after insertion:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}

return 0;
}
```


//C Program to Delete Element from nth Position of the LIST

```
#include <stdio.h>
#define MAX 10 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, position, i;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Check whether the list is empty
    if (currentSize == 0)
    {
        printf("List is Empty.\n");
        return 0;
    }

    // Read elements of the list
    printf("Enter %d elements:\n", currentSize);
    for (i = 0; i < currentSize; i++)
    {
        scanf("%d", &list[i]);
    }
}
```

```
// Read nth position to delete
printf("Enter the position to delete: ");
scanf("%d", &position);

// If nth position is not available
if (position > currentSize || position < 1)
{
    printf("nth Position Not available in List\n");
    return 0;
}

// If nth position is the last position
if (position == currentSize)
{
    // Delete last element (no shifting required)
    currentSize = currentSize - 1;
}
```

```
else
{
    // Move elements one position backward (from n+1 to current size)
    for (i = position; i < currentSize; i++)
    {
        list[i - 1] = list[i];
    }

    // Decrease current size
    currentSize = currentSize - 1;
}

// Display list after deletion
printf("List after deletion:\n");
for (i = 0; i < currentSize; i++)
{
    printf("%d ", list[i]);
}

return 0;
}
```


// C Program to Search Element in the LIST

```
#include <stdio.h>
#define MAX 10 // Maximum size of the list
int main()
{
    int list[MAX];
    int currentSize, searchElement;
    int i, position, found = 0;

    // Read current size of the list
    printf("Enter current size of the list: ");
    scanf("%d", &currentSize);

    // Check whether the list is empty
    if (currentSize == 0)
    {
        printf("List is Empty\n");
        return 0;
    }
    // Read elements of the list
    printf("Enter %d elements:\n", currentSize);
    for (i = 0; i < currentSize; i++)
    {
        scanf("%d", &list[i]);
    }
}
```

```
// Read element to search
printf("Enter element to search: ");
scanf("%d", &searchElement);

// Last Position = Current Size
int lastPosition = currentSize;

// Search from Position 1 to Last Position
for (position = 1; position <= lastPosition; position++)
{
    if (list[position - 1] == searchElement)
    {
        printf("Search Element available in Position %d\n", position);
        found = 1;
        break;
    }
}
// If element not found
if (found == 0)
{
    printf("Search Element not available in the List\n");
}
return 0;
}
```

// C Program to Print the Elements in the LIST

```
#include <stdio.h>
```

```
#define MAX 10 // Maximum size of the list
```

```
int main()
```

```
{
```

```
    int list[MAX];
```

```
    int currentSize;
```

```
    int position;
```

```
    // Read current size of the list
```

```
    printf("Enter current size of the list: ");
```

```
    scanf("%d", &currentSize);
```

```
    // Check whether the list is empty
```

```
    if (currentSize == 0)
```

```
    {
```

```
        printf("List is Empty\n");
```

```
        return 0;
```

```
    }
```

```
// Read elements of the list
printf("Enter %d elements:\n", currentSize);
for (position = 1; position <= currentSize; position++)
{
    scanf("%d", &list[position - 1]);
}

// Last Position = Current Size
int lastPosition = currentSize;

// Print elements with position
printf("Position\tElement\n");
for (position = 1; position <= lastPosition; position++)
{
    printf("%d\t\t%d\n", position, list[position - 1]);
}

return 0;
}
```

****** END OF Module-1 Additional Information ******

Dr Dattatreya P Mankame

CS1006

Data Structures

Unit – II Linked List

Unit II: Linked List

9 hours

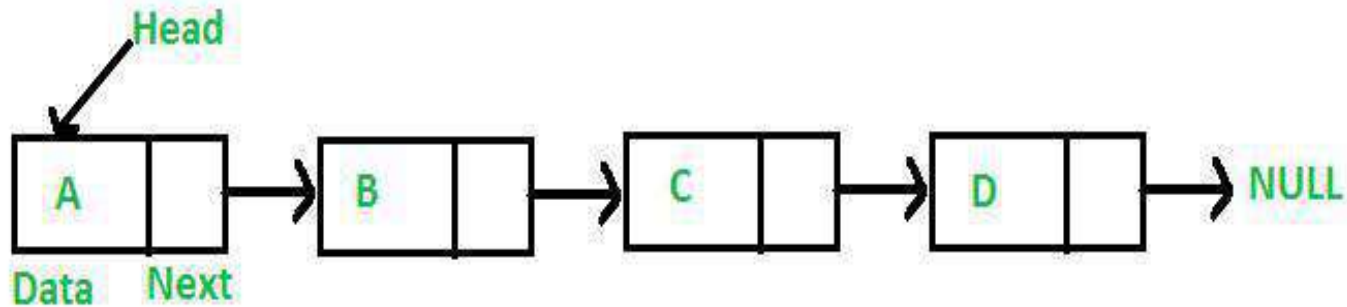
- **Linked Lists**
 - ✓ Singly,
 - ✓ Doubly,
 - ✓ Circular Singly,
 - ✓ Circular Doubly
- **Implementation of Operations –**
 - ✓ Create,
 - ✓ Insert,
 - ✓ Traverse,
 - ✓ Delete,
 - ✓ Search and
 - ✓ Update

Implementation of List ADT

- ❖ A linked list is a *sequential access* data structure, where each element can be accessed only in particular order.
- A typical illustration of sequential access is a roll of paper or tape - all prior material must be unrolled in order to get to data you want.

Linked List Implementation – List

- Like arrays, Linked List is a linear data structure.
- Unlike arrays, linked list elements are not stored at contiguous location; the elements are linked using pointers.



Need of Linked List

Arrays can be used to store linear data of similar types, but arrays have following limitations.

- ❑ The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage.
- ❑ Inserting a new element in an array of elements is expensive, because room has to be created for the new elements and to create room existing elements have to shifted.

Implications of Array

Advantages over arrays

- ✓ Dynamic size
- ✓ Ease of insertion/deletion

Drawbacks

- ✓ Random access is not allowed. We have to access elements sequentially starting from the first node. So we cannot do binary search with linked lists.
- ✓ Extra memory space for a pointer is required with each element of the list.

Representation of Linked List in 'C'

- A linked list is represented by a pointer to the first node of the linked list. The first node is called head. If the linked list is empty, then value of head is NULL.
- Each node in a list consists of at least two parts:
 - ✓ data
 - ✓ pointer to the next node
- In C, we can represent a node using structures.

Representation of Linked List in 'C'

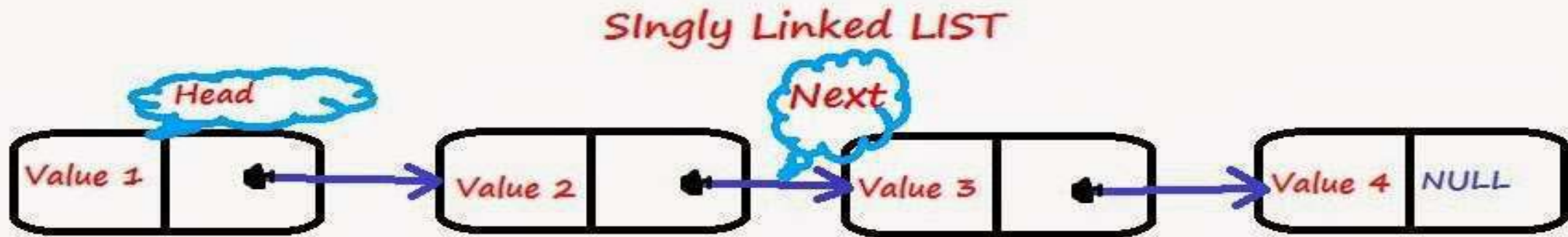
Type declaration for linked list

```
#ifndef _List_H
struct Node; //Variable declarations
typedef struct Node *PtrToNode;
typedef PtrToNode List;
typedef PtrToNode Position;

List MakeEmpty(List L); //Function Declarations
int IsEmpty(List L);
int IsLast(Position P, List L);
Position Find(ElementType X, List L);
void Delete (ElementType X, List L);
Position FindPrevious(ElementType X, List L);
void Insert(ElementType X, List L, Position P);
void DeleteList(List L);
Position Header(List L);
Position First(List L);
Position Advance(Position P);
ElementType Retrieve(Position P);

#endif /* _List_H */
//Structure of Node declaration
struct Node
{
    ElementType Element;
    Position Next;
}
```

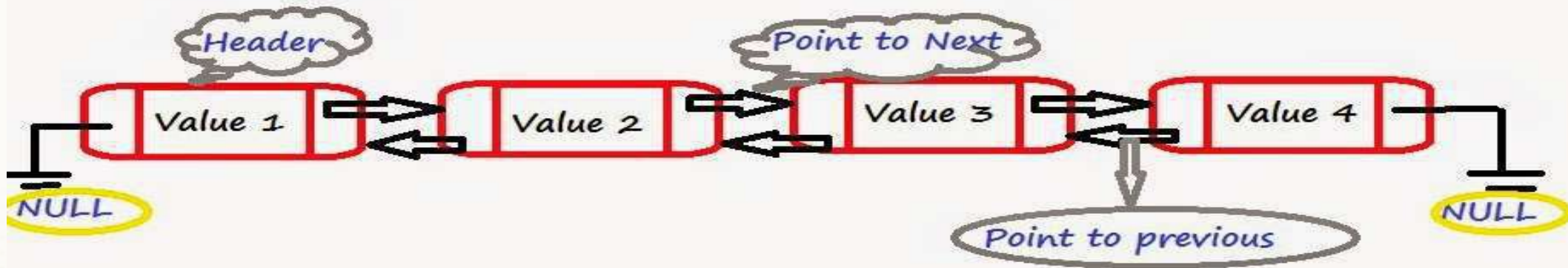

Singly Linked List



1. Each Element/Node of a list comprising of 2 items (Data, Reference to next node)
2. The last node has a reference to null.
3. The entry point into a linked list is called the head of the list. It should be noted that head is not a separate node, but the reference to the first node.
4. If the list is empty then the head is a null reference.

Doubly Linked List

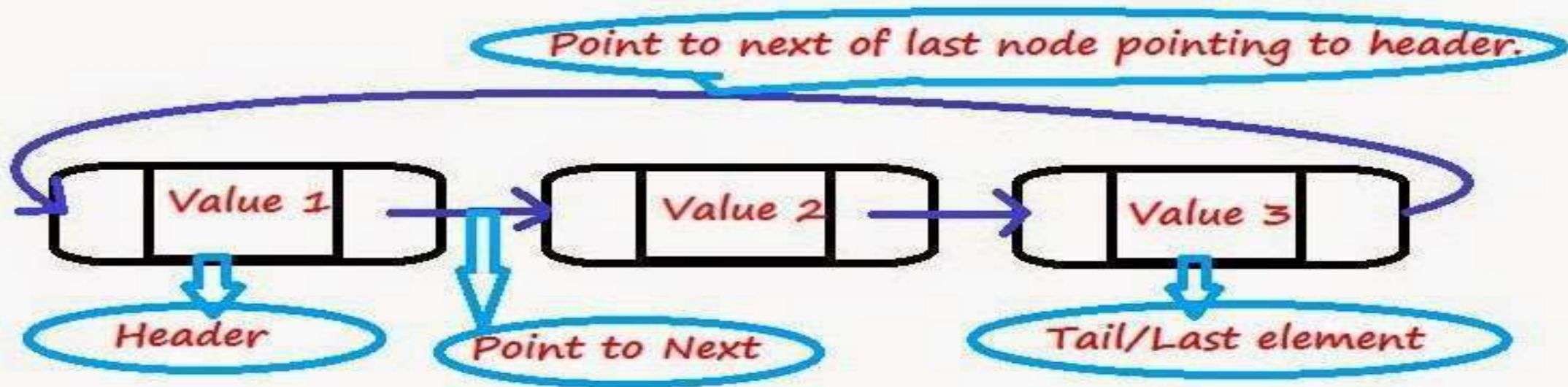
Doubly Linked LIST



1. Each node/element of a list comprising of 3 items (Data, Reference to next node & Reference to previous node)
2. The Reference to next node of a last node is NULL.
3. The Reference to previous node of a header node is NULL.
4. Two way access is possible. We can start accessing nodes from start as well as from last.

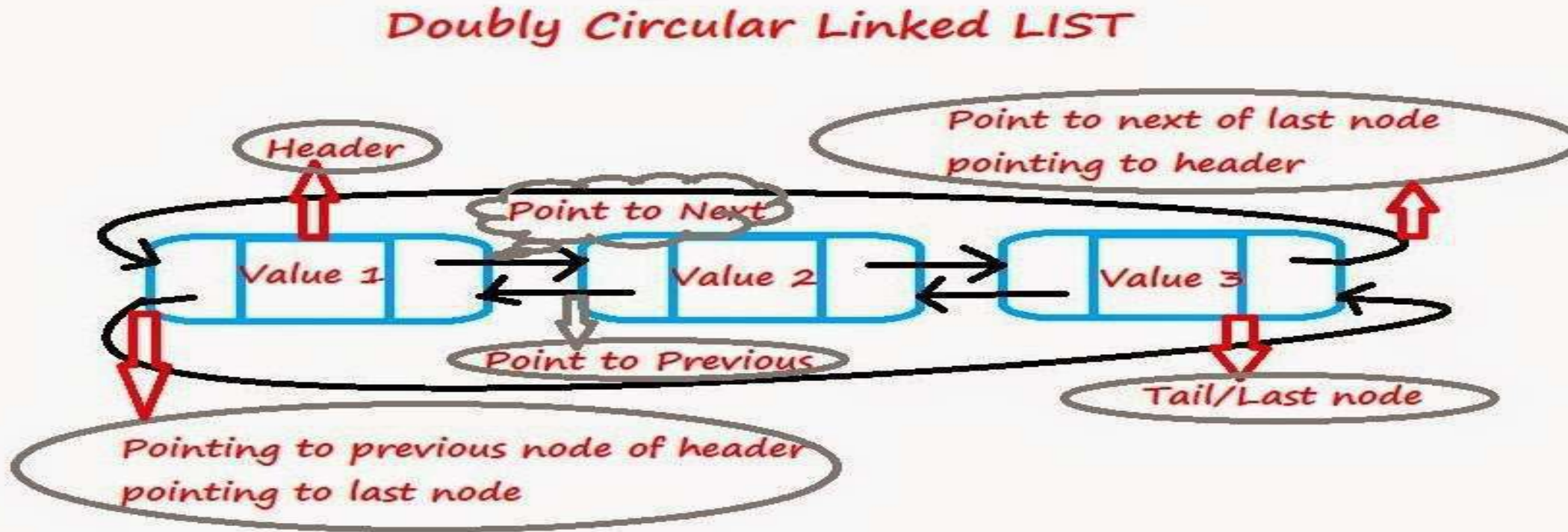
Singly Circular Linked List

Singly Circular Linked LIST



1. Same as Singly linked list
2. point to next of last node pointing to header.

Doubly Circular Linked List



1. Same as doubly linked list.
2. point to next of last node pointing to header.
3. point to previous of header pointing to last node.

Comparison

<u>S.No</u>	Array Based	Linked List
1	No Wasted Space for an individual element	Only need space for the objects actually on the list
2	Size must be predetermined; wasted space for lists with empty slots	Overhead for links(an extra pointer added to every node)
3	$\Omega(n)$ (or greater)	$\Theta(n)$

Applications of List

- Lists can be used to store a list of elements. However, unlike in traditional arrays, lists can expand and shrink, and are stored dynamically in memory.
- In computing, lists are easier to implement than sets.
- Lists also form the basis for other abstract data types including the queue, the stack, and their variations.

Polynomial Manipulation

- A polynomial is a mathematical expression consisting of a sum of terms, each term including a variable or variables raised to a power and multiplied by a coefficient. The simplest polynomials have one variable.
- **Representation of a Polynomial:** A polynomial is an expression that contains more than two terms. A term is made up of coefficient and exponent. An example of polynomial is

- $P(x) = 4x^3 + 6x^2 + 7x + 9$

$$f(x) = \sum_{i=0}^n a_i x^i$$

Polynomial Manipulation

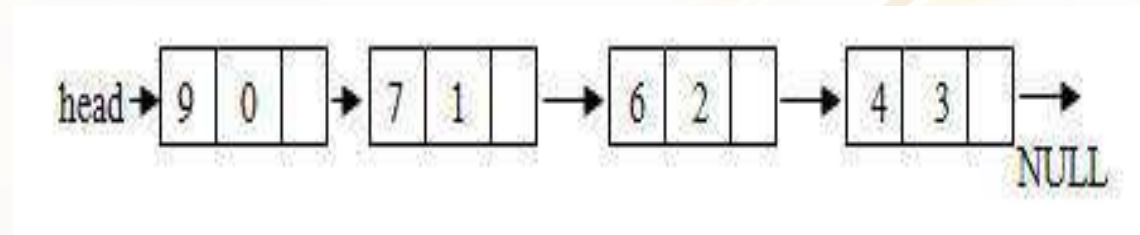
- A polynomial thus may be represented using arrays or linked lists.
- Array representation assumes that the exponents of the given expression are arranged from 0 to the highest value (degree), which is represented by the subscript of the array beginning with 0.
- The coefficients of the respective exponent are placed at an appropriate index in the array.

arr	9	7	6	4	(coefficients)
	0	1	2	3	(exponents)

Polynomial Structure

- A polynomial may also be represented using a linked list. A structure may be defined such that it contains two parts- one is the coefficient and second is the corresponding exponent. The structure definition may be given as shown below:

```
struct polynomial
{
    int coefficient;
    int exponent;
    struct polynomial *next;
};
```



Polynomial Operations

The most common operations on a polynomial:

Add & Subtract

Multiply

Differentiate

Integrate

etc...

There are different ways of implementing the polynomial ADT:

Array (not recommended)

Linked List (preferred and recommended)

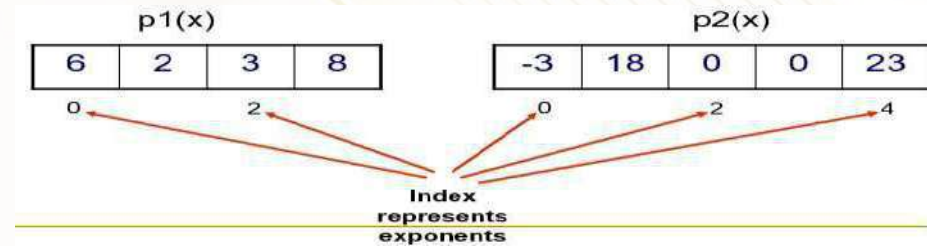
Array Implementation:

$$p1(x) = 8x^3 + 3x^2 + 2x + 6$$

$$p2(x) = 23x^4 + 18x - 3$$

This is why arrays aren't good to represent polynomials:

$$p3(x) = 16x^{21} - 3x^5 + 2x + 6$$



Polynomial using Arrays

Advantages of using an Array:

- ❖ only good for non-sparse polynomials.
- ❖ ease of storage and retrieval.

Disadvantages of using an Array:

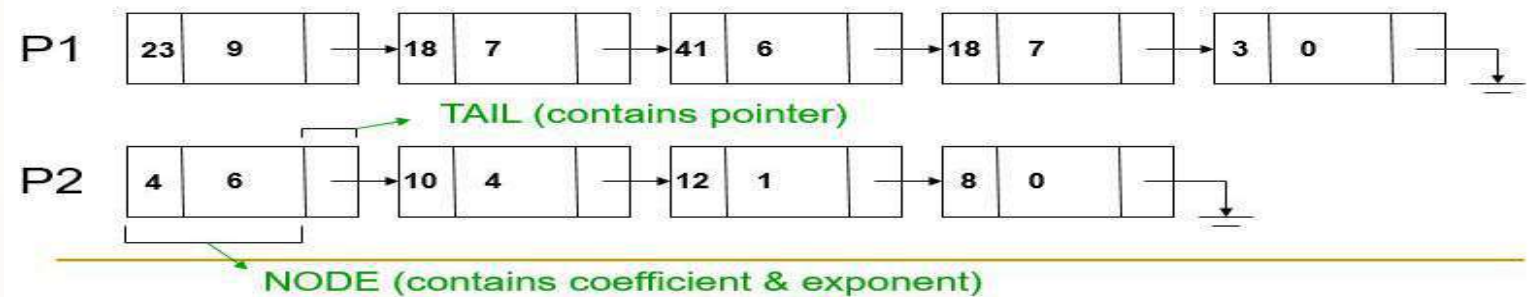
- ❖ Have to allocate array size ahead of time. Huge array size required for sparse polynomials.
- ❖ Waste of space and runtime.

Polynomial using Linked List

$$p1(x) = 23x^9 + 18x^7 + 41x^6 + 163x^4 + 3$$

$$p2(x) = 4x^6 + 10x^4 + 12x + 8$$

```
struct polynode {  
    int coef;  
    int exp;  
    struct polynode * next;  
};  
typedef struct polynode *polyptr;
```



Polynomial using Linked List

Advantages of using a Linked list:

- save space (don't have to worry about sparse polynomials) and easy to maintain
- don't need to allocate list size and can declare nodes (terms) only as needed

Disadvantages of using a Linked list :

- can't go backwards through the list
- can't jump to the beginning of the list from the end.

Polynomial Addition

For adding two polynomials using arrays is straightforward method, since both the arrays may be added up element wise beginning from 0 to n-1, resulting in addition of two polynomials.

Addition of two polynomials using linked list requires comparing the exponents, and wherever the exponents are found to be same, the coefficients are added up.

For terms with different exponents, the complete term is simply added to the result thereby making it a part of addition result.

$$\begin{array}{r} x^5 + 3x^3 + 4 \\ + 6x^6 + \quad + 4x^3 \\ \hline 6x^6 + x^5 + 7x^3 + 4 \end{array}$$

Polynomial Addition

- Adding polynomials using a Linked list representation: (storing the result in p3)
- To do this, we have to break the process down to cases:

Case 1: exponent of p1 > exponent of p2

- Copy node of p1 to end of p3.
- [go to next node]

Case 2: exponent of p1 < exponent of p2

- Copy node of p2 to end of p3.
- [go to next node]

Case 3: exponent of p1 = exponent of p2

- Create a new node in p3 with the same exponent and with the sum of the coefficients of p1 and p2.

Polynomial Multiplication

- Multiplication of two polynomials however requires manipulation of each node such that the exponents are added up and the coefficients are multiplied.
- After each term of first polynomial is operated upon with each term of the second polynomial, then the result has to be added up by comparing the exponents and adding the coefficients for similar exponents and including terms as such with dissimilar exponents in the result.

Multiply Polynomials

$$(x + 2)(x^2 + 3x + 4)$$

$$x^3 + 3x^2 + 4x + 2x^2 + 6x + 8$$
$$= x^3 + 5x^2 + 10x + 8$$

Polynomial operations

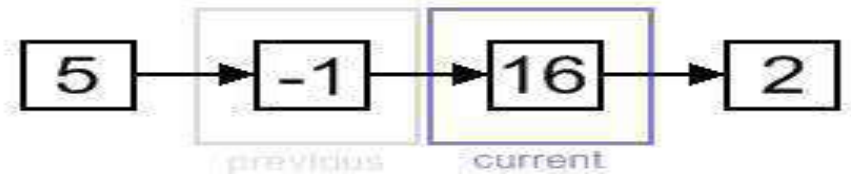
- ✓ Insertion
- ✓ Deletion
- ✓ Merge two sorted linked lists
- ✓ Traversal

- Assume, that we have a list with some nodes. Traversal is the very basic operation, which presents as a part in almost every operation on a singly-linked list.
- For instance, algorithm may traverse a singly-linked list to find a value, find a position for insertion, etc. For a singly-linked list, only forward direction traversal is possible.

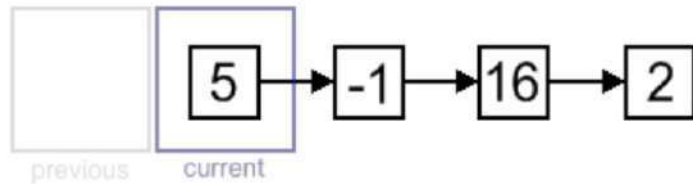
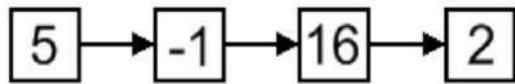
Traversal algorithm

- Beginning from the head, check, if the end of a list hasn't been reached yet;
- do some actions with the current node, which is specific for particular algorithm; current node becomes previous and –next node becomes current. Go to the step 1.

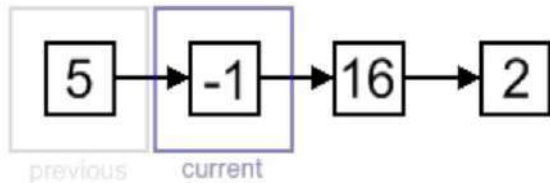
Polynomial Operations



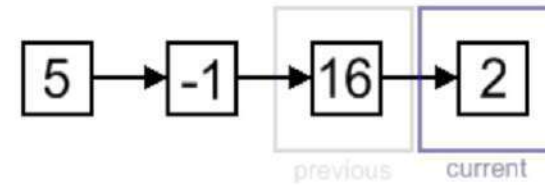
step 2
sum = 4



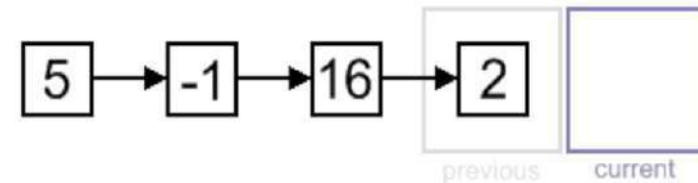
initial state
sum = 0



step 1
sum = 5



step 3
sum = 20



final state
sum = 22

External References

1. <https://nptel.ac.in/courses/106102064/>
2. <https://nptel.ac.in/courses/106103069/>
3. <https://nptel.ac.in/courses/106106127/>
4. <https://nptel.ac.in/courses/106106130/>
5. <https://www.geeksforgeeks.org/data-structures/>
6. <https://www.javatpoint.com/data-structure-tutorial>
7. <https://www.coursera.org/learn/data-structures>
8. <https://www.studytonight.com/data-structures/introduction-to-data-structures>
9. <https://www.youtube.com/watch?v=egb6qFw42JM> (Tamil videos)

Thank You

C Program: Create and Display a Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>

// Define structure for Node
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

// Function to insert node at the end
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = createNode(value);

    if (head == NULL) {
        return newNode;
    }

    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    return head;
}

// Function to display the linked list
void displayList(struct Node* head) {
    struct Node* temp = head;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
    }
}
```

```

        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data for node %d: ", i + 1);
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    displayList(head);

    return 0;
}

```

C Program: Insert at Beginning in Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Node
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at beginning
struct Node* insertAtBeginning(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = head; // Point new node to current head
    head = newNode;      // Make new node as head

    return head;
}

// Function to display the list
void displayList(struct Node* head) {
    struct Node* temp = head;

```

```

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data to insert at beginning: ");
        scanf("%d", &value);
        head = insertAtBeginning(head, value);
    }

    displayList(head);

    return 0;
}

```

C Program: Insert at End in Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Node
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;

```



```

newNode->next = NULL;

// If list is empty
if (head == NULL) {
    return newNode;
}

// Traverse to last node
struct Node* temp = head;
while (temp->next != NULL) {
    temp = temp->next;
}

temp->next = newNode;
return head;
}

// Function to display list
void displayList(struct Node* head) {
    struct Node* temp = head;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data to insert at end: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    displayList(head);

    return 0;
}

```

C Program: Insert at Any Position

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define structure
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
// Function to insert at any position
```

```
struct Node* insertAtPosition(struct Node* head, int value, int position) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    if (newNode == NULL) {  
        printf("Memory allocation failed!\n");  
        exit(1);  
    }
```

```
    newNode->data = value;  
    newNode->next = NULL;
```

```
// Insert at beginning
```

```
if (position == 1) {  
    newNode->next = head;  
    return newNode;  
}
```

```
struct Node* temp = head;
```

```
// Traverse to (position - 1)
```

```
for (int i = 1; i < position - 1 && temp != NULL; i++) {  
    temp = temp->next;  
}
```

```
if (temp == NULL) {  
    printf("Invalid Position!\n");  
    free(newNode);  
    return head;  
}
```

```
newNode->next = temp->next;  
temp->next = newNode;
```

```
return head;
```

```
}
```

```
// Function to display list
```

```
void displayList(struct Node* head) {  
    struct Node* temp = head;
```

```
    if (head == NULL) {  
        printf("List is empty.\n");
```

```

        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, position;

    printf("Enter number of initial nodes: ");
    scanf("%d", &n);

    // Create initial list by inserting
    for (int i = 0; i < n; i++) {
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        scanf("%d", &value);
        newNode->data = value;
        newNode->next = head;
        head = newNode;
    }

    printf("Enter value to insert: ");
    scanf("%d", &value);

    printf("Enter position: ");
    scanf("%d", &position);

    head = insertAtPosition(head, value, position);

    displayList(head);

    return 0;
}

```

C Program: Delete Front Node from Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {

```



```

struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

if (newNode == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
}

newNode->data = value;
newNode->next = NULL;

if (head == NULL)
    return newNode;

struct Node* temp = head;
while (temp->next != NULL)
    temp = temp->next;

temp->next = newNode;
return head;
}

// Function to delete front node
struct Node* deleteFront(struct Node* head)
{
    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;
    printf("Deleted element: %d\n", temp->data);

    head = head->next; // Move head to next node
    free(temp);       // Free old first node

    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    head = deleteFront(head);

    display(head);

    return 0;
}

```

C Program: Delete Last Node from Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

```

```

    temp->next = newNode;
    return head;
}

// Function to delete last node
struct Node* deleteLast(struct Node* head) {
    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    // If only one node
    if (head->next == NULL) {
        printf("Deleted element: %d\n", head->data);
        free(head);
        return NULL;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    // Traverse to last node
    while (temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }

    printf("Deleted element: %d\n", temp->data);
    prev->next = NULL;
    free(temp);

    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;

```

```

int n, value;

printf("Enter number of nodes: ");
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    printf("Enter data: ");
    scanf("%d", &value);
    head = insertAtEnd(head, value);
}

display(head);

head = deleteLast(head);

display(head);

return 0;
}
C Program: Delete Node at Any Position from Singly Linked List
#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value)
{
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}

```



```

// Function to delete node at given position
struct Node* deleteAtPosition(struct Node* head, int position) {
    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;

    // Delete first node
    if (position == 1) {
        printf("Deleted element: %d\n", temp->data);
        head = temp->next;
        free(temp);
        return head;
    }

    struct Node* prev = NULL;

    // Traverse to desired position
    for (int i = 1; i < position && temp != NULL; i++) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid position!\n");
        return head;
    }

    printf("Deleted element: %d\n", temp->data);
    prev->next = temp->next;
    free(temp);

    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, position;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter position to delete: ");
    scanf("%d", &position);

    head = deleteAtPosition(head, position);

    display(head);

    return 0;
}

```

C Program: Search an Element in Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;
}

```

```

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}

// Function to search element
void search(struct Node* head, int key) {
    struct Node* temp = head;
    int position = 1;
    int found = 0;

    while (temp != NULL) {
        if (temp->data == key) {
            printf("Element %d found at position %d\n", key, position);
            found = 1;
            break;
        }
        temp = temp->next;
        position++;
    }

    if (!found) {
        printf("Element %d not found in the list.\n", key);
    }
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

```

```

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter element to search: ");
    scanf("%d", &key);

    search(head, key);

    return 0;
}

```

C Program: Print All Elements in Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* next;
};

// Function to insert at end (for creating list)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}

// Function to print all elements
void printList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
    }
}

```



```

        return;
    }

    struct Node* temp = head;

    printf("Elements in the Linked List: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    printList(head);

    return 0;
}

```

C Program: Create and Display a doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Doubly Linked List Node
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;

```

```

newNode->prev = NULL;
newNode->next = NULL;

// If list is empty
if (head == NULL) {
    return newNode;
}

struct Node* temp = head;

// Traverse to last node
while (temp->next != NULL)
{
    temp = temp->next;
}

temp->next = newNode;
newNode->prev = temp;

return head;
}

// Function to display list (Forward)
void displayForward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List (Forward): ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Function to display list (Backward)
void displayBackward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;

    // Move to last node
    while (temp->next != NULL) {
        temp = temp->next;
    }

    printf("Doubly Linked List (Backward): ");

```

```

while (temp != NULL) {
    printf("%d <-> ", temp->data);
    temp = temp->prev;
}
printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    displayForward(head);
    displayBackward(head);

    return 0;
}

```

C Program: Insert at Beginning in doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Doubly Linked List Node
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at beginning
struct Node* insertAtBeginning(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL) {
        head->prev = newNode;
    }
}

```

```

    }

    head = newNode;
    return head;
}

// Function to display forward
void displayForward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List (Forward): ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data to insert at beginning: ");
        scanf("%d", &value);
        head = insertAtBeginning(head, value);
    }

    displayForward(head);

    return 0;
}

```

C Program: Insert at End in doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure for Doubly Linked List Node
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end

```



```

struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    // If list is empty
    if (head == NULL) {
        return newNode;
    }

    struct Node* temp = head;

    // Traverse to last node
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to display list (Forward)
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

```

```

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    return 0;
}

```

C Program: Insert at Any Position in doubly linked list

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for initial creation)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to insert at any position
struct Node* insertAtPosition(struct Node* head, int value, int position) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

```

```

if (newNode == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
}

newNode->data = value;

// Insert at beginning
if (position == 1) {
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
        head->prev = newNode;

    return newNode;
}

struct Node* temp = head;

// Traverse to (position - 1)
for (int i = 1; i < position - 1 && temp != NULL; i++) {
    temp = temp->next;
}

if (temp == NULL) {
    printf("Invalid Position!\n");
    free(newNode);
    return head;
}

newNode->next = temp->next;
newNode->prev = temp;

if (temp->next != NULL)
    temp->next->prev = newNode;

temp->next = newNode;

return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {

```

```

        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, position;

    printf("Enter number of initial nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter value to insert: ");
    scanf("%d", &value);

    printf("Enter position: ");
    scanf("%d", &position);

    head = insertAtPosition(head, value, position);

    display(head);

    return 0;
}

```

C Program: Delete Front Node from doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
}

```



```

    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to delete front node
struct Node* deleteFront(struct Node* head)
{
    if (head == NULL)
    {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;
    printf("Deleted element: %d\n", temp->data);

    head = head->next; // Move head to next node

    if (head != NULL)
        head->prev = NULL;

    free(temp);
    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
}

```

```

    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    head = deleteFront(head);

    display(head);

    return 0;
}

```

C Program: Delete Last Node from doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for testing)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;
}

```

```

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to delete last node
struct Node* deleteLast(struct Node* head)
{
    if (head == NULL)
    {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;

    // If only one node
    if (temp->next == NULL) {
        printf("Deleted element: %d\n", temp->data);
        free(temp);
        return NULL;
    }

    // Traverse to last node
    while (temp->next != NULL)
        temp = temp->next;

    printf("Deleted element: %d\n", temp->data);
    temp->prev->next = NULL;          // Remove link
    free(temp);

    return head;
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    head = deleteLast(head);

    display(head);

    return 0;
}

```

C Program: Delete Node at Any Position from doubly linked list

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for creating list)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;

```



```

while (temp->next != NULL)
    temp = temp->next;

temp->next = newNode;
newNode->prev = temp;

return head;
}

// Function to delete node at any position
struct Node* deleteAtPosition(struct Node* head, int position) {

    if (head == NULL) {
        printf("List is empty! Cannot delete.\n");
        return NULL;
    }

    struct Node* temp = head;

    // Delete first node
    if (position == 1) {
        printf("Deleted element: %d\n", temp->data);
        head = temp->next;

        if (head != NULL)
            head->prev = NULL;

        free(temp);
        return head;
    }

    // Traverse to desired position
    for (int i = 1; i < position && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid position!\n");
        return head;
    }

    printf("Deleted element: %d\n", temp->data);

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    if (temp->prev != NULL)
        temp->prev->next = temp->next;

    free(temp);

    return head;
}

```

```

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, position;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter position to delete: ");
    scanf("%d", &position);

    head = deleteAtPosition(head, position);

    display(head);

    return 0;
}

```

C Program: Search an Element in doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

```

```

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

```

```

// Function to insert at end (for creating list)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to search element
void search(struct Node* head, int key) {
    struct Node* temp = head;
    int position = 1;
    int found = 0;

    while (temp != NULL) {
        if (temp->data == key) {
            printf("Element %d found at position %d\n", key, position);
            found = 1;
            break;
        }
        temp = temp->next;
        position++;
    }

    if (!found) {
        printf("Element %d not found in the list.\n", key);
    }
}

// Function to display list
void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
}

```

```

    }

    struct Node* temp = head;
    printf("Doubly Linked List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

    display(head);

    printf("Enter element to search: ");
    scanf("%d", &key);

    search(head, key);

    return 0;
}

```

C Program: Print All Elements in doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Define structure
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// Function to insert at end (for creating list)
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
}

```



```

    }

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL)
        return newNode;

    struct Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

// Function to print all elements (Forward Traversal)
void displayForward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;
    printf("Doubly Linked List (Forward): ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Function to print all elements (Backward Traversal)
void displayBackward(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node* temp = head;

    // Move to last node
    while (temp->next != NULL)
        temp = temp->next;

    printf("Doubly Linked List (Backward): ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
}

```

```

    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &value);
        head = insertAtEnd(head, value);
    }

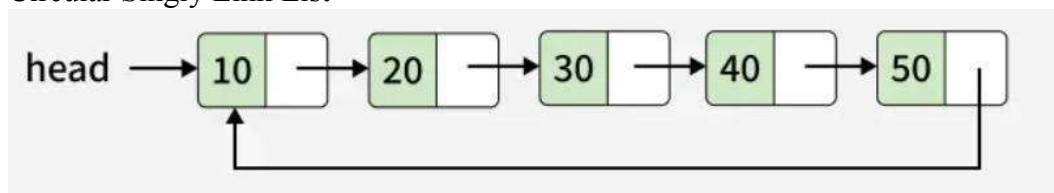
    displayForward(head);
    displayBackward(head);

    return 0;
}

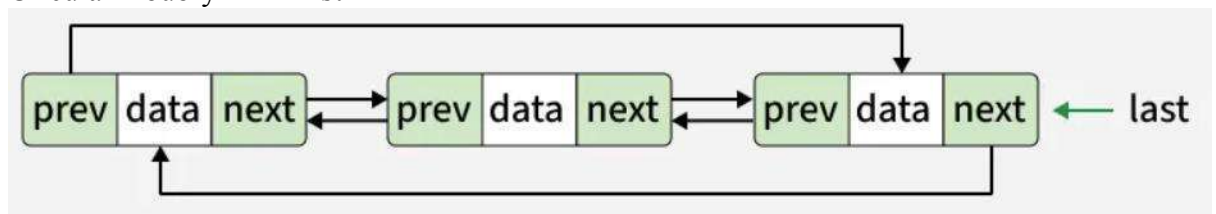
```

Circular Link List:

1. Circular Singly Link List



2. Circular Doubly Link List



Advantage of Circular Linked List

- Efficient Traversal
- No Null Pointers / References
- Useful for Repetitive Tasks
- Uniform Traversal
- Efficient Memory Utilization

Disadvantage of Circular Linked List

- Complex Implementation
- Infinite Loop Risk
- Harder to Debug
- Deletion Complexity

Applications:

1. Round Robin Scheduling
2. Multi-Player Games
3. Implementation of Circular Queues
4. Real-time Systems Like, Traffic Light Control
5. Image/Slide Show Viewers
6. Token Passing in Networks (Ring Topology)

C Program to Create and Display a Circular Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>

// Structure definition
struct node
{
    int data;
    struct node *next;
};

struct node *head = NULL, *temp, *newnode;

// Function to create circular linked list
void create()
{
    int n, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        newnode = (struct node*)malloc(sizeof(struct node));

        printf("Enter data for node %d: ", i);
        scanf("%d", &newnode->data);

        if(head == NULL)
        {
            head = newnode;
            temp = newnode;
            newnode->next = head; // Circular link
        }
        else
        {
            temp->next = newnode;
            newnode->next = head; // Circular link
            temp = newnode;
        }
    }
}

// Function to display circular linked list
void display()
{

```

```

struct node *ptr;

if(head == NULL)
{
printf("List is empty\n");
return;
}

ptr = head;

printf("Circular Linked List:\n");

do
{
printf("%d -> ", ptr->data);
ptr = ptr->next;
} while(ptr != head);

printf("(Back to Head)\n");
}

// Main Function
int main()
{
create();
display();

return 0;
}

```

C Program: Insert at Beginning in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Structure definition
struct node
{
int data;
struct node *next;
};

struct node *head = NULL;

// Function to insert at beginning
void insert_beginning(int value)
{
struct node *newnode, *temp;

newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = value;

// If list is empty
if(head == NULL)
{
head = newnode;

```



```

newnode->next = head;
}
else
{
temp = head;

// Move to last node
while(temp->next != head)
temp = temp->next;

newnode->next = head;
temp->next = newnode;
head = newnode;
}
}

// Function to display list
void display()
{
struct node *temp;

if(head == NULL)
{
printf("List is empty\n");
return;
}

temp = head;

printf("Circular Linked List:\n");

do
{
printf("%d -> ", temp->data);
temp = temp->next;
}
while(temp != head);

printf("HEAD\n");
}

// Main function
int main()
{
int n, i, value;

printf("Enter number of nodes: ");
scanf("%d", &n);

for(i = 1; i <= n; i++)
{
printf("Enter value %d: ", i);
scanf("%d", &value);

insert_beginning(value);

```

```
}
```

```
display();
```

```
return 0;
```

```
}
```

C Program: Insert at End in a Circular Singly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Structure definition
```

```
struct node
```

```
{
```

```
int data;
```

```
struct node *next;
```

```
};
```

```
struct node *head = NULL;
```

```
// Function to insert at end
```

```
void insert_end(int value)
```

```
{
```

```
struct node *newnode, *temp;
```

```
newnode = (struct node*)malloc(sizeof(struct node));
```

```
newnode->data = value;
```

```
// If list is empty
```

```
if(head == NULL)
```

```
{
```

```
head = newnode;
```

```
newnode->next = head;
```

```
}
```

```
else
```

```
{
```

```
temp = head;
```

```
// Move to last node
```

```
while(temp->next != head)
```

```
{
```

```
temp = temp->next;
```

```
}
```

```
temp->next = newnode;
```

```
newnode->next = head;
```

```
}
```

```
}
```

```
// Function to display list
```

```
void display()
```

```
{
```

```
struct node *temp;
```

```
if(head == NULL)
```

```

{
printf("List is empty\n");
return;
}

temp = head;

printf("Circular Linked List:\n");

do
{
printf("%d -> ", temp->data);
temp = temp->next;
}
while(temp != head);

printf("HEAD\n");
}

// Main function
int main()
{
int n, i, value;

printf("Enter number of nodes: ");
scanf("%d", &n);

for(i = 1; i <= n; i++)
{
printf("Enter value %d: ", i);
scanf("%d", &value);

insert_end(value);
}

display();

return 0;
}

```

C Program: Insert at Any position in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

// Structure definition
struct node
{
int data;
struct node *next;
};

struct node *head = NULL;

// Function to insert at any position
void insert_position(int value, int pos)

```

```

{
struct node *newnode, *temp;
int i;

newnode = (struct node*)malloc(sizeof(struct node));
newnode->data = value;

// Insert at beginning
if(pos == 1)
{
if(head == NULL)
{
head = newnode;
newnode->next = head;
}
else
{
temp = head;

while(temp->next != head)
temp = temp->next;

newnode->next = head;
temp->next = newnode;
head = newnode;
}
}
else
{
temp = head;

for(i = 1; i < pos-1; i++)
temp = temp->next;

newnode->next = temp->next;
temp->next = newnode;
}

}

// Function to display list
void display()
{
struct node *temp;

if(head == NULL)
{
printf("List is empty\n");
return;
}

temp = head;

printf("Circular Linked List:\n");

do

```



```

{
printf("%d -> ", temp->data);
temp = temp->next;
}
while(temp != head);

printf("HEAD\n");
}

// Main function
int main()
{
int n, i, value, pos;

printf("Enter number of nodes: ");
scanf("%d",&n);

for(i=1;i<=n;i++)
{
printf("Enter value: ");
scanf("%d",&value);

printf("Enter position: ");
scanf("%d",&pos);

insert_position(value,pos);
}

display();

return 0;
}

```

C Program: Delete a node at Beginning in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
int data;
struct Node* next;
};

```

```

struct Node* head = NULL;

```

```

/* Create Circular Singly Linked List */

```

```

void create(int n) {
struct Node *temp, *newNode;
int data, i;

```

```

if (n <= 0) {
printf("Invalid number of nodes.\n");
return;
}

```

```

printf("Enter data for node 1: ");

```

```

scanf("%d", &data);

head = (struct Node*)malloc(sizeof(struct Node));
if (head == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

head->data = data;
head->next = head; // Circular link
temp = head;

for (i = 2; i <= n; i++) {
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

printf("Enter data for node %d: ", i);
scanf("%d", &data);

newNode->data = data;
newNode->next = head;
temp->next = newNode;
temp = newNode;
}
}

/* Delete Node at Beginning */
void deleteBeginning() {
struct Node *temp, *last;

// Condition 1: List Empty
if (head == NULL) {
printf("\nList is empty. Nothing to delete.\n");
return;
}

// Condition 2: Only One Node
if (head->next == head) {
temp = head;
head = NULL;
free(temp);
printf("\nOnly node deleted. List is now empty.\n");
return;
}

// Condition 3: Multiple Nodes
last = head;

// Traverse to last node
while (last->next != head) {

```

```

last = last->next;
}

temp = head; // Store old head
head = head->next; // Move head forward
last->next = head; // Maintain circular link

free(temp);

printf("\nFirst node deleted successfully.\n");
}

/* Display Circular List */
void display() {
    struct Node *temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Circular Linked List: ");

    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);

    printf("(Back to Head)\n");
}

/* Main Function */
int main() {
    int n;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    create(n);

    printf("\nBefore deletion:\n");
    display();

    deleteBeginning();

    printf("\nAfter deletion:\n");
    display();

    return 0;
}

```

C Program: Delete a node at End in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* head = NULL;

/* Create Circular Singly Linked List */
void create(int n) {
struct Node *temp, *newNode;
int data, i;

if (n <= 0) {
printf("Invalid number of nodes.\n");
return;
}

printf("Enter data for node 1: ");
scanf("%d", &data);

head = (struct Node*)malloc(sizeof(struct Node));
if (head == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

head->data = data;
head->next = head; // Circular link
temp = head;

for (i = 2; i <= n; i++) {
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

printf("Enter data for node %d: ", i);
scanf("%d", &data);

newNode->data = data;
newNode->next = head;
temp->next = newNode;
temp = newNode;
}

/* Delete Node at End */
void deleteEnd() {
struct Node *curr, *prev;

// Condition 1: Empty list

```



```

if (head == NULL) {
printf("\nList is empty. Nothing to delete.\n");
return;
}

// Condition 2: Only one node
if (head->next == head) {
free(head);
head = NULL;
printf("\nOnly node deleted. List is now empty.\n");
return;
}

// Condition 3: Multiple nodes
prev = NULL;
curr = head;

// Traverse to last node
while (curr->next != head) {
prev = curr;
curr = curr->next;
}

prev->next = head; // Maintain circular link
free(curr);

printf("\nLast node deleted successfully.\n");
}

/* Display Circular Linked List */
void display() {
struct Node *temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head;
printf("Circular Linked List: ");

do {
printf("%d -> ", temp->data);
temp = temp->next;
} while (temp != head);

printf("(Back to Head)\n");
}

/* Main Function */
int main() {
int n;

printf("Enter number of nodes: ");
scanf("%d", &n);

```

```

create(n);

printf("\nBefore deletion:\n");
display();

deleteEnd();

printf("\nAfter deletion:\n");
display();

return 0;
}

```

C Program: Delete a node at any position in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* head = NULL;

/* Create Circular Singly Linked List */
void create(int n) {
struct Node *temp, *newNode;
int data, i;

if (n <= 0) {
printf("Invalid number of nodes.\n");
return;
}

printf("Enter data for node 1: ");
scanf("%d", &data);

head = (struct Node*)malloc(sizeof(struct Node));
if (head == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

head->data = data;
head->next = head;
temp = head;

for (i = 2; i <= n; i++) {
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

printf("Enter data for node %d: ", i);

```

```

scanf("%d", &data);

newNode->data = data;
newNode->next = head;
temp->next = newNode;
temp = newNode;
}
}

/* Count Nodes */
int countNodes() {
    int count = 0;
    struct Node* temp = head;

    if (head == NULL)
        return 0;

    do {
        count++;
        temp = temp->next;
    } while (temp != head);

    return count;
}

/* Delete Node at Any Position */
void deleteAtPosition(int pos) {
    struct Node *temp, *prev, *last;
    int count, i;

    // Condition 1: Empty list
    if (head == NULL) {
        printf("\nList is empty. Nothing to delete.\n");
        return;
    }

    count = countNodes();

    // Condition 2: Invalid position
    if (pos <= 0 || pos > count) {
        printf("\nInvalid position.\n");
        return;
    }

    // Condition 3: Delete first node
    if (pos == 1) {

        // Only one node
        if (head->next == head) {
            free(head);
            head = NULL;
            printf("\nOnly node deleted. List is now empty.\n");
            return;
        }
    }
}

```

```

// More than one node
last = head;
while (last->next != head)
last = last->next;

temp = head;
head = head->next;
last->next = head;
free(temp);

printf("\nNode at position 1 deleted successfully.\n");
return;
}

// Condition 4 & 5: Delete middle or last node
prev = head;
for (i = 1; i < pos - 1; i++) {
prev = prev->next;
}

temp = prev->next;
prev->next = temp->next;

free(temp);

printf("\nNode at position %d deleted successfully.\n", pos);
}

/* Display Circular Linked List */
void display() {
struct Node *temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head;
printf("Circular Linked List: ");

do {
printf("%d -> ", temp->data);
temp = temp->next;
} while (temp != head);

printf("(Back to Head)\n");
}

/* Main Function */
int main() {
int n, pos;

printf("Enter number of nodes: ");
scanf("%d", &n);

```

```

create(n);

printf("\nBefore deletion:\n");
display();

printf("\nEnter position to delete: ");
scanf("%d", &pos);

deleteAtPosition(pos);

printf("\nAfter deletion:\n");
display();

return 0;
}

```

C Program: Search an Element in a Circular Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
};

struct Node* head = NULL;

/* Create Circular Singly Linked List */
void create(int n) {
struct Node *temp, *newNode;
int data, i;

if (n <= 0) {
printf("Invalid number of nodes.\n");
return;
}

printf("Enter data for node 1: ");
scanf("%d", &data);

head = (struct Node*)malloc(sizeof(struct Node));
if (head == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

head->data = data;
head->next = head; // Circular link
temp = head;

for (i = 2; i <= n; i++) {
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
exit(0);
}

```



```

}

printf("Enter data for node %d: ", i);
scanf("%d", &data);

newNode->data = data;
newNode->next = head;
temp->next = newNode;
temp = newNode;
}
}

/* Search Element */
void search(int key) {
    struct Node* temp;
    int position = 1;
    int found = 0;

    // Condition 1: Empty list
    if (head == NULL) {
        printf("\nList is empty.\n");
        return;
    }

    temp = head;

    // Traverse circular list
    do {
        if (temp->data == key) {
            printf("\nElement %d found at position %d.\n", key, position);
            found = 1;
            break;
        }
        temp = temp->next;
        position++;
    } while (temp != head);

    // Condition 5: Not found
    if (!found) {
        printf("\nElement %d not found in the list.\n", key);
    }
}

/* Display Circular Linked List */
void display() {
    struct Node *temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Circular Linked List: ");

```

```

do {
printf("%d -> ", temp->data);
temp = temp->next;
} while (temp != head);

printf("(Back to Head)\n");
}

/* Main Function */
int main() {
int n, key;

printf("Enter number of nodes: ");
scanf("%d", &n);

create(n);

printf("\nList elements:\n");
display();

printf("\nEnter element to search: ");
scanf("%d", &key);

search(key);

return 0;
}

```

C Program: Insert at Beginning in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
struct Node* prev;
};

struct Node* head = NULL;

/* Insert at Beginning */
void insertBeginning(int data) {
struct Node* newNode;
struct Node* last;

// Allocate memory
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
return;
}

newNode->data = data;

// Condition 1: Empty list

```

```

if (head == NULL) {
    newNode->next = newNode;
    newNode->prev = newNode;
    head = newNode;
    printf("Node inserted as the only node in the list.\n");
    return;
}

// Condition 2: One or more nodes
last = head->prev; // Last node

newNode->next = head;
newNode->prev = last;

head->prev = newNode;
last->next = newNode;

head = newNode;

printf("Node inserted at the beginning successfully.\n");
}

/* Display Forward */
void displayForward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head->prev; // Start from last
    printf("Backward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
}

```

```

printf("(Back to Last)\n");
}

/* Main Function */
int main() {
int n, data, i;

printf("Enter number of nodes to insert at beginning: ");
scanf("%d", &n);

for (i = 1; i <= n; i++) {
printf("Enter data for node %d: ", i);
scanf("%d", &data);
insertBeginning(data);
}

printf("\nAfter insertion:\n");
displayForward();
displayBackward();

return 0;
}

```

C Program: Insert at End in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
int data;
struct Node* next;
struct Node* prev;
};

struct Node* head = NULL;

/* Insert at End */
void insertEnd(int data) {
struct Node* newNode;
struct Node* last;

// Allocate memory
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
return;
}

newNode->data = data;

// Condition 1: Empty list
if (head == NULL) {
newNode->next = newNode;
newNode->prev = newNode;
head = newNode;
printf("Node inserted as the only node in the list.\n");
return;
}

```

```

}

// Condition 2: One or more nodes
last = head->prev; // Last node

newNode->next = head;
newNode->prev = last;

last->next = newNode;
head->prev = newNode;

printf("Node inserted at the end successfully.\n");
}

/* Display Forward */
void displayForward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head->prev; // Start from last
    printf("Backward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
    printf("(Back to Last)\n");
}

/* Main Function */
int main() {
    int n, data, i;

    printf("Enter number of nodes to insert at end: ");

```



```

scanf("%d", &n);

for (i = 1; i <= n; i++) {
    printf("Enter data for node %d: ", i);
    scanf("%d", &data);
    insertEnd(data);
}

printf("\nAfter insertion:\n");
displayForward();
displayBackward();

return 0;
}

```

C Program: Insert at Any position in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

struct Node* head = NULL;

/* Count Nodes */
int countNodes() {
    int count = 0;
    struct Node* temp = head;

    if (head == NULL)
        return 0;

    do {
        count++;
        temp = temp->next;
    } while (temp != head);

    return count;
}

/* Insert at Any Position */
void insertAtPosition(int data, int pos) {
    struct Node *newNode, *temp, *last;
    int count, i;

    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = data;

```

```

count = countNodes();

// Condition 1: Empty list
if (head == NULL) {
    if (pos != 1) {
        printf("Invalid position. List is empty.\n");
        free(newNode);
        return;
    }
    newNode->next = newNode;
    newNode->prev = newNode;
    head = newNode;
    printf("Node inserted as the only node in the list.\n");
    return;
}

// Condition 2: Invalid position
if (pos <= 0 || pos > count + 1) {
    printf("Invalid position.\n");
    free(newNode);
    return;
}

// Condition 3: Insert at Beginning
if (pos == 1) {
    last = head->prev;

    newNode->next = head;
    newNode->prev = last;

    last->next = newNode;
    head->prev = newNode;

    head = newNode;

    printf("Node inserted at beginning successfully.\n");
    return;
}

// Condition 4: Insert at End
if (pos == count + 1) {
    last = head->prev;

    newNode->next = head;
    newNode->prev = last;

    last->next = newNode;
    head->prev = newNode;

    printf("Node inserted at end successfully.\n");
    return;
}

// Condition 5: Insert at Middle
temp = head;

```

```

for (i = 1; i < pos - 1; i++) {
temp = temp->next;
}

newNode->next = temp->next;
newNode->prev = temp;

temp->next->prev = newNode;
temp->next = newNode;

printf("Node inserted at position %d successfully.\n", pos);
}

/* Display Forward */
void displayForward() {
struct Node* temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head;
printf("Forward: ");
do {
printf("%d <-> ", temp->data);
temp = temp->next;
} while (temp != head);
printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
struct Node* temp;

if (head == NULL) {
printf("List is empty.\n");
return;
}

temp = head->prev;
printf("Backward: ");
do {
printf("%d <-> ", temp->data);
temp = temp->prev;
} while (temp != head->prev);
printf("(Back to Last)\n");
}

/* Main Function */
int main() {
int n, data, pos, i;

printf("Enter number of initial nodes: ");
scanf("%d", &n);

```

```

for (i = 1; i <= n; i++) {
printf("Enter data for node %d (insert at end): ", i);
scanf("%d", &data);
insertAtPosition(data, i); // inserting sequentially
}

printf("\nCurrent List:\n");
displayForward();

printf("\nEnter data to insert: ");
scanf("%d", &data);

printf("Enter position to insert: ");
scanf("%d", &pos);

insertAtPosition(data, pos);

printf("\nUpdated List:\n");
displayForward();
displayBackward();

return 0;
}

```

C Program: Delete a node at Beginning in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
int data;
struct Node* next;
struct Node* prev;
};

```

```

struct Node* head = NULL;

```

```

/* Delete Node at Beginning */
void deleteBeginning() {
struct Node *temp, *last;

```

```

// Condition 1: Empty list
if (head == NULL) {
printf("List is empty. Nothing to delete.\n");
return;
}

```

```

// Condition 2: Only one node
if (head->next == head) {
free(head);
head = NULL;
printf("Only node deleted. List is now empty.\n");
return;
}

```

```

// Condition 3: Multiple nodes

```

```

temp = head;
last = head->prev;

head = head->next; // Move head forward
head->prev = last;
last->next = head;

free(temp);

printf("First node deleted successfully.\n");
}

/* Insert at End (Helper for Testing) */
void insertEnd(int data) {
    struct Node* newNode;
    struct Node* last;

    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = data;

    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
        return;
    }

    last = head->prev;

    newNode->next = head;
    newNode->prev = last;

    last->next = newNode;
    head->prev = newNode;
}

/* Display Forward */
void displayForward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("\n");
}

```



```

} while (temp != head);
printf("(Back to Head)\n");
}

```

```

/* Display Backward */
void displayBackward() {
    struct Node* temp;

```

```

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

```

```

    temp = head->prev;
    printf("Backward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
    printf("(Back to Last)\n");
}

```

```

/* Main Function */
int main() {
    int n, data, i;

```

```

    printf("Enter number of nodes: ");
    scanf("%d", &n);

```

```

    for (i = 1; i <= n; i++) {
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);
        insertEnd(data);
    }

```

```

    printf("\nBefore deletion:\n");
    displayForward();

```

```

    deleteBeginning();

```

```

    printf("\nAfter deletion:\n");
    displayForward();
    displayBackward();

```

```

    return 0;
}

```

C Program: Delete a node at End in a Circular Doubly Linked List

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};

```

```

struct Node* head = NULL;

/* Delete Node at End */
void deleteEnd() {
    struct Node *last, *secondLast;

    // Condition 1: Empty list
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }

    // Condition 2: Only one node
    if (head->next == head) {
        free(head);
        head = NULL;
        printf("Only node deleted. List is now empty.\n");
        return;
    }

    // Condition 3: Multiple nodes
    last = head->prev;
    secondLast = last->prev;

    secondLast->next = head;
    head->prev = secondLast;

    free(last);

    printf("Last node deleted successfully.\n");
}

/* Insert at End (Helper for Testing) */
void insertEnd(int data) {
    struct Node* newNode;
    struct Node* last;

    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = data;

    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
        return;
    }

    last = head->prev;

```

```
newNode->next = head;
newNode->prev = last;
```

```
last->next = newNode;
head->prev = newNode;
}
```

```
/* Display Forward */
void displayForward() {
    struct Node* temp;
```

```
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
```

```
    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}
```

```
/* Display Backward */
void displayBackward() {
    struct Node* temp;
```

```
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
```

```
    temp = head->prev;
    printf("Backward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    } while (temp != head->prev);
    printf("(Back to Last)\n");
}
```

```
/* Main Function */
int main() {
    int n, data, i;
```

```
    printf("Enter number of nodes: ");
    scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++) {
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);
        insertEnd(data);
    }
```

```
printf("\nBefore deletion:\n");
displayForward();
```

```
deleteEnd();
```

```
printf("\nAfter deletion:\n");
displayForward();
displayBackward();
```

```
return 0;
}
```

C Program: Delete a node at any position in a Circular Doubly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};
```

```
struct Node* head = NULL;
```

```
/* Count Nodes */
```

```
int countNodes() {
    int count = 0;
    struct Node* temp = head;
```

```
    if (head == NULL)
        return 0;
```

```
    do {
        count++;
        temp = temp->next;
    } while (temp != head);
```

```
    return count;
}
```

```
/* Delete at Any Position */
```

```
void deleteAtPosition(int pos) {
    struct Node *temp, *last;
    int count, i;
```

```
    // Condition 1: Empty list
```

```
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }
```

```
    count = countNodes();
```

```
    // Condition 2: Invalid position
```

```
    if (pos <= 0 || pos > count) {
```

```

printf("Invalid position.\n");
return;
}

// Condition 3: Only one node
if (head->next == head) {
free(head);
head = NULL;
printf("Only node deleted. List is now empty.\n");
return;
}

// Condition 4: Delete at Beginning
if (pos == 1) {
temp = head;
last = head->prev;

head = head->next;
head->prev = last;
last->next = head;

free(temp);
printf("Node at position 1 deleted successfully.\n");
return;
}

// Condition 5: Delete at End
if (pos == count) {
temp = head->prev; // Last node
temp->prev->next = head; // Second last next = head
head->prev = temp->prev; // Update head prev

free(temp);
printf("Last node deleted successfully.\n");
return;
}

// Condition 6: Delete at Middle
temp = head;
for (i = 1; i < pos; i++) {
temp = temp->next;
}

temp->prev->next = temp->next;
temp->next->prev = temp->prev;

free(temp);
printf("Node at position %d deleted successfully.\n", pos);
}

/* Insert at End (Helper for Testing) */
void insertEnd(int data) {
struct Node* newNode;
struct Node* last;

```



```
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
return;
}
```

```
newNode->data = data;
```

```
if (head == NULL) {
newNode->next = newNode;
newNode->prev = newNode;
head = newNode;
return;
}
```

```
last = head->prev;
```

```
newNode->next = head;
newNode->prev = last;
```

```
last->next = newNode;
head->prev = newNode;
}
```

```
/* Display Forward */
void displayForward() {
struct Node* temp;
```

```
if (head == NULL) {
printf("List is empty.\n");
return;
}
```

```
temp = head;
printf("Forward: ");
do {
printf("%d <-> ", temp->data);
temp = temp->next;
} while (temp != head);
printf("(Back to Head)\n");
}
```

```
/* Main Function */
int main() {
int n, data, pos, i;
```

```
printf("Enter number of nodes: ");
scanf("%d", &n);
```

```
for (i = 1; i <= n; i++) {
printf("Enter data for node %d: ", i);
scanf("%d", &data);
insertEnd(data);
}
```

```
printf("\nBefore deletion:\n");
displayForward();
```

```
printf("\nEnter position to delete: ");
scanf("%d", &pos);
```

```
deleteAtPosition(pos);
```

```
printf("\nAfter deletion:\n");
displayForward();
```

```
return 0;
}
```

C Program: Search an Element in a Circular Doubly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
int data;
struct Node* next;
struct Node* prev;
};
```

```
struct Node* head = NULL;
```

```
/* Insert at End (Helper Function) */
```

```
void insertEnd(int data) {
struct Node* newNode;
struct Node* last;
```

```
newNode = (struct Node*)malloc(sizeof(struct Node));
if (newNode == NULL) {
printf("Memory allocation failed.\n");
return;
}
```

```
newNode->data = data;
```

```
// Empty list
```

```
if (head == NULL) {
newNode->next = newNode;
newNode->prev = newNode;
head = newNode;
return;
}
```

```
last = head->prev;
```

```
newNode->next = head;
newNode->prev = last;
```

```
last->next = newNode;
head->prev = newNode;
}
```

```

/* Search Element */
void search(int key) {
    struct Node* temp;
    int position = 1;
    int found = 0;

    // Condition 1: Empty list
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;

    // Traverse circular doubly linked list
    do {
        if (temp->data == key) {
            printf("Element %d found at position %d.\n", key, position);
            found = 1;
            break;
        }
        temp = temp->next;
        position++;
    } while (temp != head);

    // Condition 5: Not found
    if (!found) {
        printf("Element %d not found in the list.\n", key);
    }
}

/* Display Forward */
void displayForward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    printf("Forward: ");
    do {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("(Back to Head)\n");
}

/* Display Backward */
void displayBackward() {
    struct Node* temp;

    if (head == NULL) {
        printf("List is empty.\n");
    }
}

```

```
return;  
}
```

```
temp = head->prev;  
printf("Backward: ");  
do {  
    printf("%d <-> ", temp->data);  
    temp = temp->prev;  
} while (temp != head->prev);  
printf("(Back to Last)\n");  
}
```

```
/* Main Function */
```

```
int main() {  
    int n, data, key, i;
```

```
    printf("Enter number of nodes: ");  
    scanf("%d", &n);
```

```
    for (i = 1; i <= n; i++) {  
        printf("Enter data for node %d: ", i);  
        scanf("%d", &data);  
        insertEnd(data);  
    }
```

```
    printf("\nList elements:\n");  
    displayForward();
```

```
    printf("\nEnter element to search: ");  
    scanf("%d", &key);
```

```
    search(key);
```

```
    return 0;  
}
```